



TRAILHEAD
TECHNOLOGY PARTNERS

Avoidifying Over-Complexification

Rooting Out Over-Engineering
in Your Projects



Jonathan "J." Tower



TRAILHEAD
TECHNOLOGY PARTNERS



TRAILHEAD
TECHNOLOGY PARTNERS

credit: scienceworld.scholastic.com

10 Types of Over-Engineering & 10 Rules to Help Avoid It

Jonathan "J." Tower



Founding Partner

- 🏆 13x Microsoft MVP in .NET & AI
- ⚙️ VP of .NET Foundation
- 🎙️ Blue Blazes podcast host



Mission-Critical Code: What

trailheadtechnology.com/connect/?t=power-of-ten&e=the-cloud-ai-summit-2026



FROM J. TOWER'S SESSION

Thanks for Coming. Here's Everything from the Session.

Mission-Critical Code: What NASA's Power of Ten Can Teach Us

 **Get the Slides + Code**
Slides and demo code, in one GitHub repo

 **Connect on LinkedIn**
New talks, Blue Blazes episodes, what I'm learning

 [Rate this session for The Cloud & AI Summit 2026](#)



Let's Talk About Your Questions.
Grab 30 minutes with me. Ask me anything you like.

PICK A TIME

Next you'll choose a slot on my calendar

Not Ready to Meet? Let's Stay in Touch.

I post what I'm learning, industry trends, plus new talks and Blue Blazes episodes.



Jonathan "J." Tower
13x Microsoft MVP in .NET and AI | VP of .NET Foundation |
Founding Partner at Trailhead

+ Follow

Jonathan "J." Tower



Why Worry About Over-Engineering

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Solving the Wrong
Problems



New Tool vs
Right Tool



Often Lead to
Rewrite



Harder to
Maintain

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Solving the Wrong
Problems



New Tool vs
Right Tool



Often Lead to
Rewrite



Harder to
Maintain



Increased
Costs



Extended
Dev Time



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



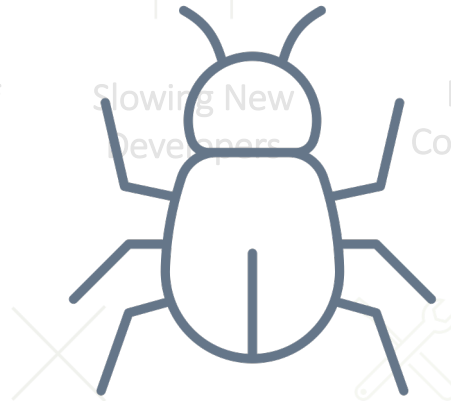
Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



**Higher Risk
Of Bugs**

Solving the
Problems

Tool vs
Right Tool



Often Lead to
Rewrite



Harder to
Maintain



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Slowing New
Developers



Often Lead to
Rewrite



Harder to
Maintain



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Solving the
Right Problem



Solving the
Right Problem

**Loss of Focus
on Features**



Often Lead to
Rewrite



Harder to
Maintain



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Solving Problems
Right Tool

**Reduced Testing
Exposure**



Often Lead to
Rewrite



Harder to
Maintain

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Solving the Wrong
Problems

**Not
Agile**



New Tool vs
Right Tool



Often Lead to
Rewrite



Harder to
Maintain



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



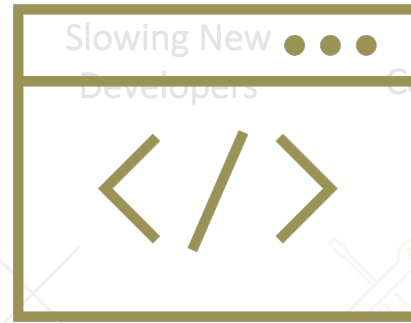
Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



Often Lead to
Rewrite

Solving to Fit
Problems



Often Lead to
Rewrite



Harder to
Maintain



TRAILHEAD
TECHNOLOGY PARTNERS

The Issues With Over-Engineering



Increased Costs



Extended
Dev Time



Higher Risk Of
Bugs



Slowing New
Developers



Higher
Complexity



Loss of Focus
on Features



Not Relying on
Outside Experts



Reduced Testing
Exposure



Not
Agile



**Harder to
Maintain**

Solving the Wrong Problems vs
Solving the Right Problems



Often Lead to
Rewrite



Harder to
Maintain

Why We Over-Engineer



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Curiosity or
Enthusiasm

Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap
**Perceived
Prestige**



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Ref
inement



Anticipation of Future
Needs



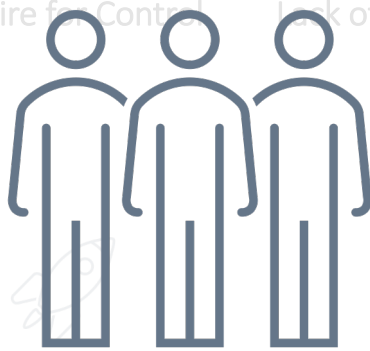
Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High Perf
Domains



Preemptive Solution
to Rare Issues

Peer Pressure /
Trends

Fear of Missing
Out

Marketing and
Hype



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously

Perceived Unique Requirements



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Attempted Perf Optimization

Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of
Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



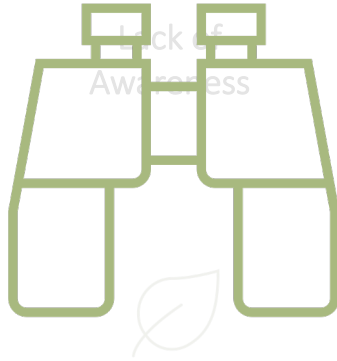
Pressure from
Stakeholders



Overconfidence



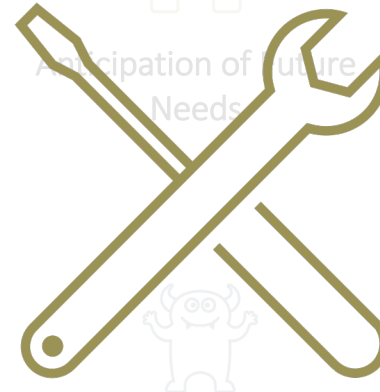
Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues

**Anticipation of
Future Needs**

**Fear of
Rework**



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues

**Pressure from
Stakeholders**



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



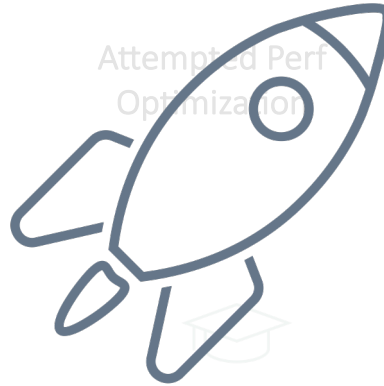
Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues

Over- Compensation



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Influence of High-Perf Domains

Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



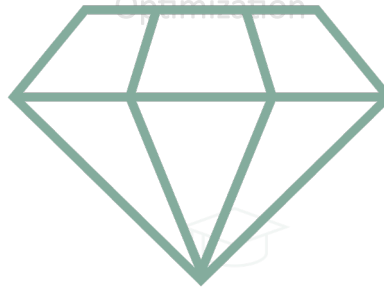
Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues

Preemptive Solution to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of
Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously

Avoid Vendor Lock-In



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues

Future Proofing



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS



Curiosity or
Enthusiasm



Perceived
Prestige



Peer Pressure
/ Trends



Fear of
Missing Out



Marketing
and Hype



Personal
Interest



Fear of
Code Smells



Perceived Unique
Requirements



Desire for Control



Lack of
Awareness



Attempted Perf
Optimization



Anticipation of Future
Needs



Fear of Rework



Pressure from
Stakeholders



Overconfidence



Over-
Compensation



Fear of Simplicity



**Burned
Previously**

Educational Gap



Fear of Future/Past
Perf Issues



Influence
of High-Perf
Domains



Preemptive Solution
to Rare Issues



Avoid Vendor
Lock-In



Showing Off



Future Proofing



Personal Interest



Burned
Previously



TRAILHEAD
TECHNOLOGY PARTNERS

10 Common **Types** of Over-Engineering

Gold-Plating

Over-Engineering Type 1



TRAILHEAD
TECHNOLOGY PARTNERS

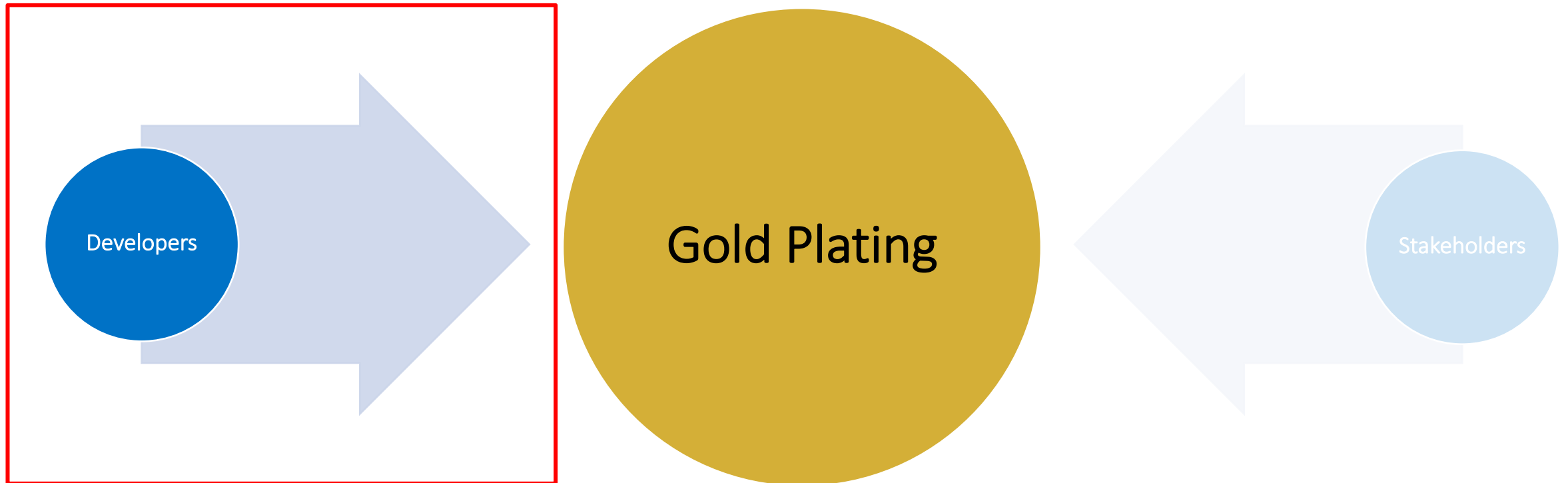
Sources of Gold-Plating



Sources of Gold-Plating

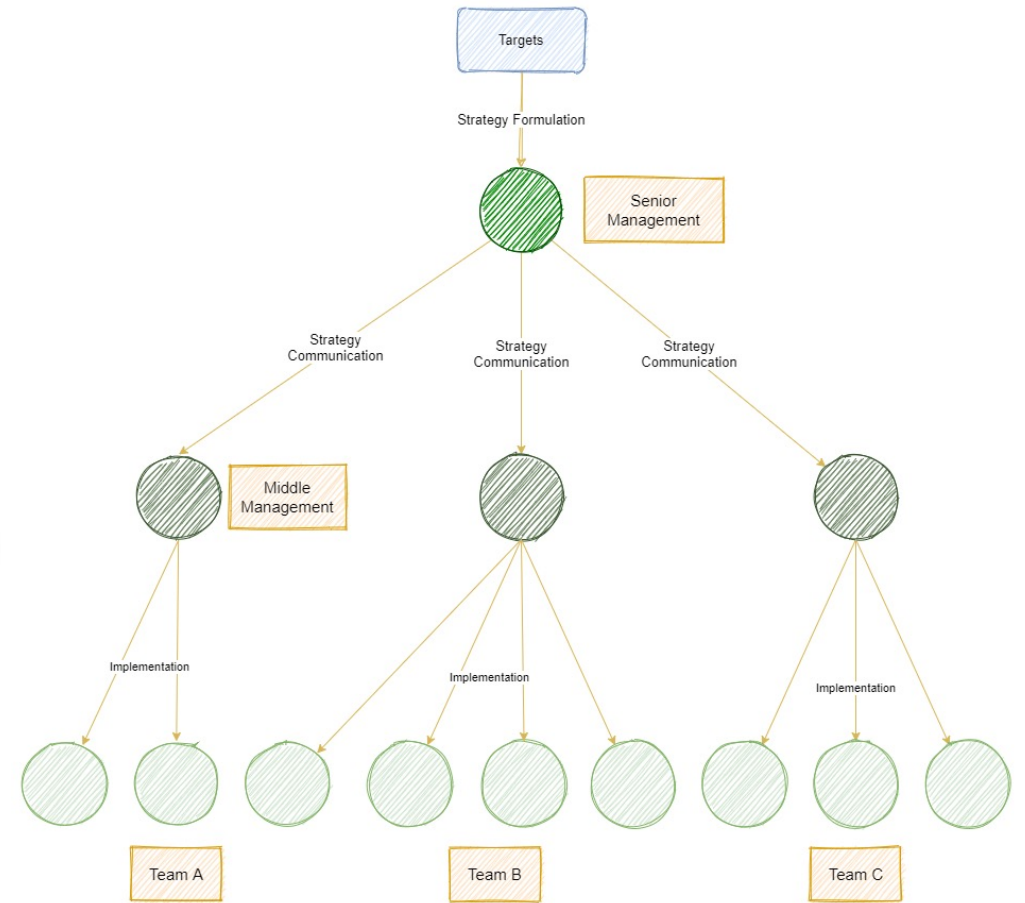


Sources of Gold-Plating



OO Gymnastics

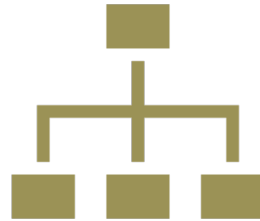
Over-Engineering Type 2



Types of OO Gymnastics



Unnecessary
Generics



Complex Inheritance
Hierarchies

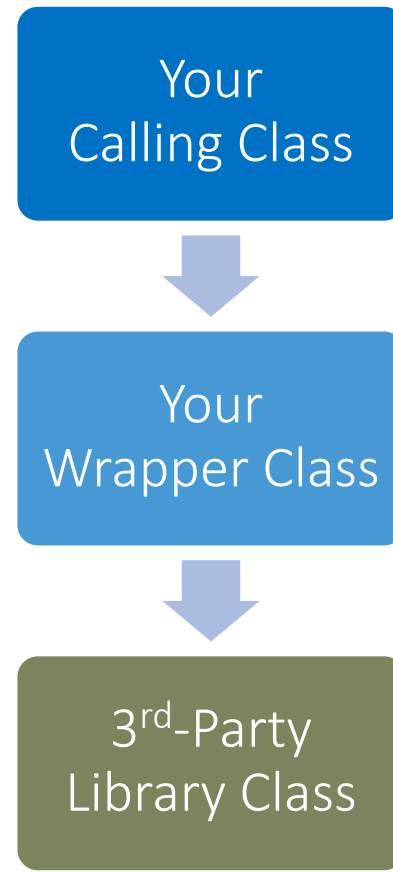


Properties Instead
of Fields

Over-Abstraction

Over-Engineering Type 3

Over-Abstraction



Why You Don't Need Over-Abstraction

- Embrace all 3rd-party library has to offer
- Ensure to select libraries that:
 1. Are small
 2. Isolated
 3. Uncomplicated
 4. Replaceable



Mattias Karlsson (he/him)
@devlead

Follow



One way to avoid vendor lock-in is to embrace all that the vendor has to offer, but ensure things are small, isolated, uncomplicated, and replaceable.

7:41 AM · Jul 21, 2022

Swapping a Library

- **Then:** weeks of rework

Swapping a Library

- **Then:** weeks of rework
- **With AI:** an afternoon with an agent

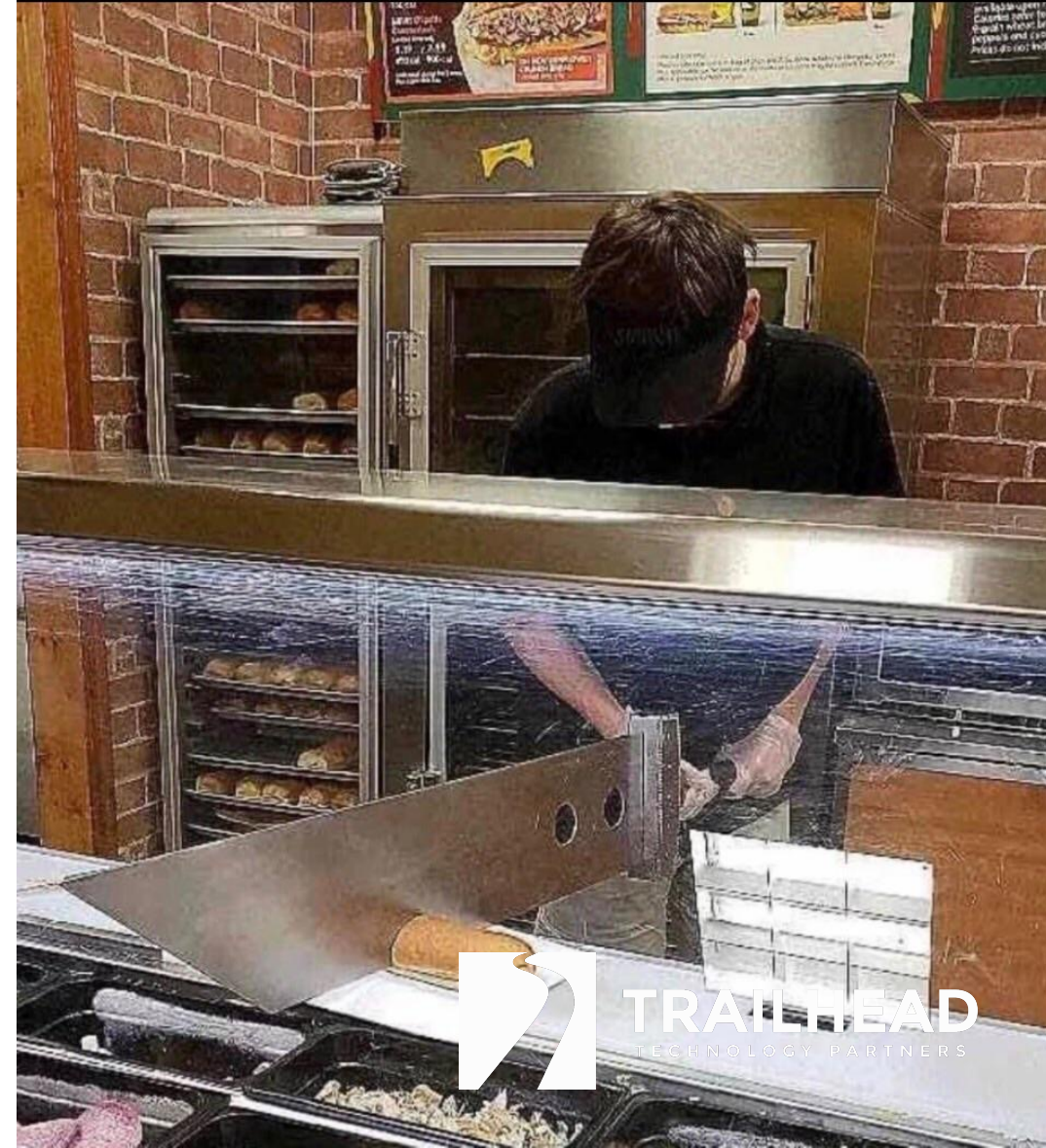
Swapping a Library

- **Then:** weeks of rework
- **With AI:** an afternoon with an agent
- **Isolate** it, don't **wrap** it

Over-Built Scalability

Over-Engineering Type 4

**When you decide
to use all your special attacks
on that level 1 monster**



Scalability Rule of Thumb

“Build your software for, at most, 1-2 orders of magnitude more than you currently need.”

- J. Tower

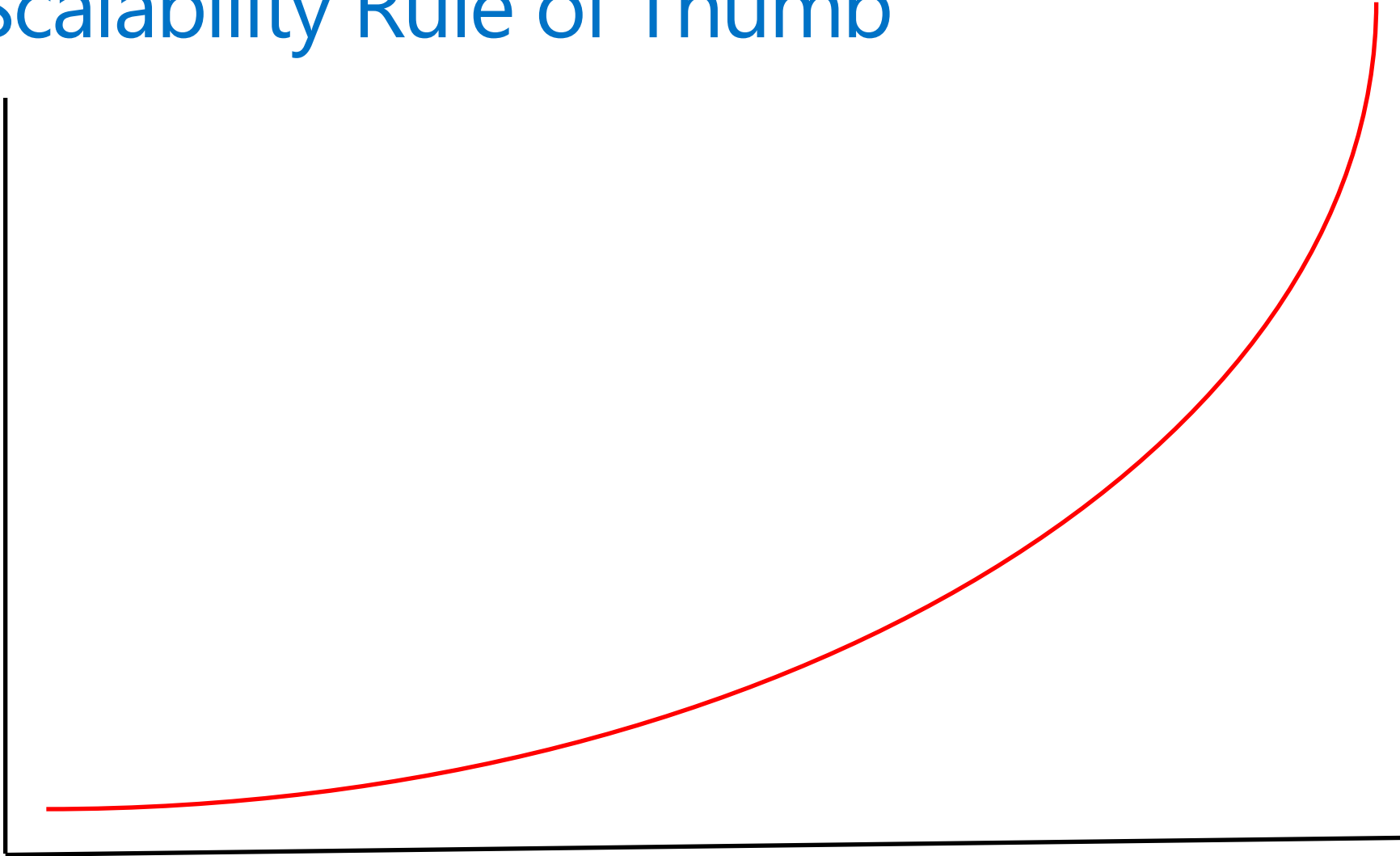
Scalability Rule of Thumb

10 $\rightarrow$ 100 - 1,000

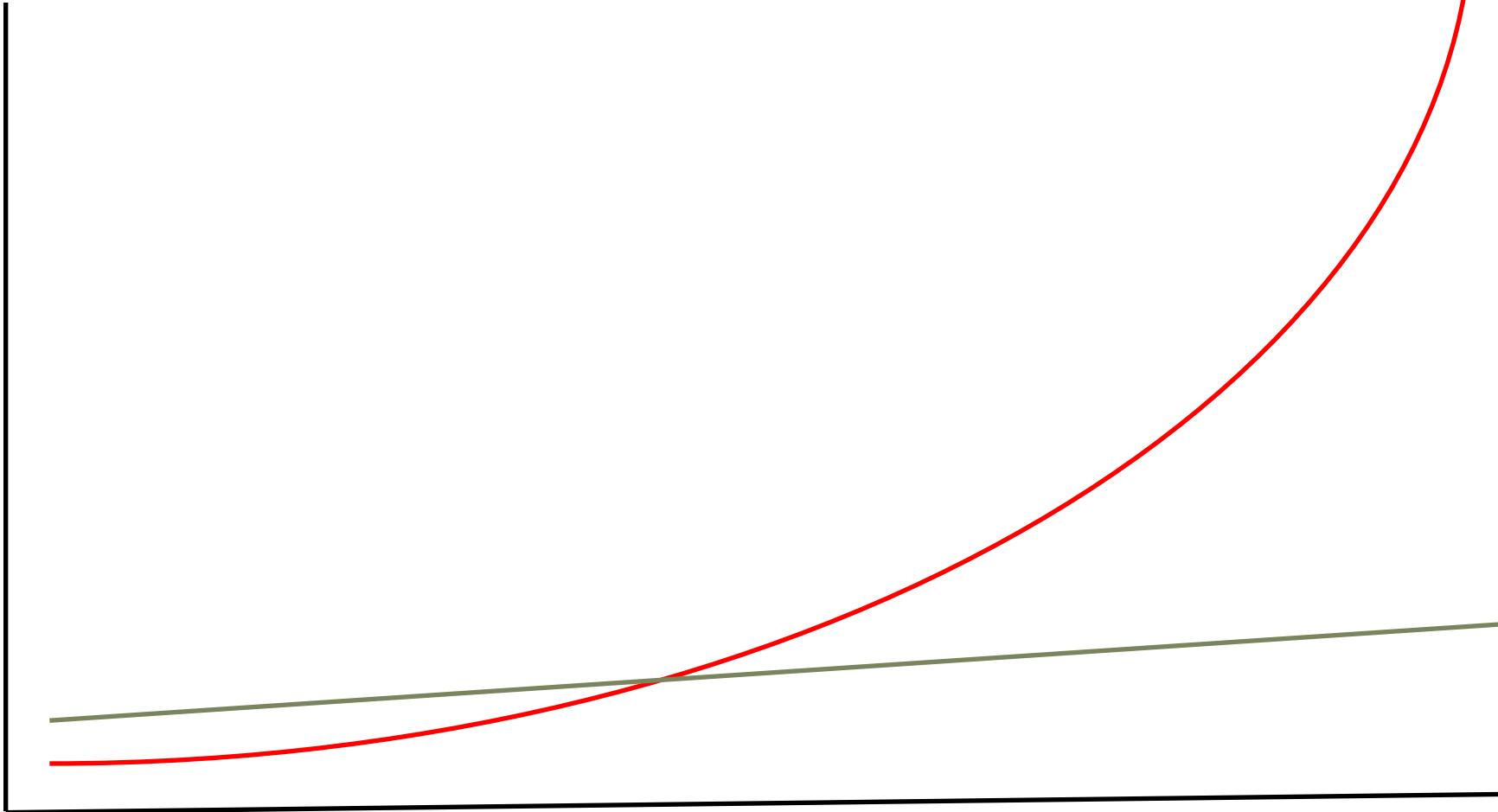
100 $\rightarrow$ 1,000 - 10,000

1,000 $\rightarrow$ 10,000 - 100,000

Scalability Rule of Thumb

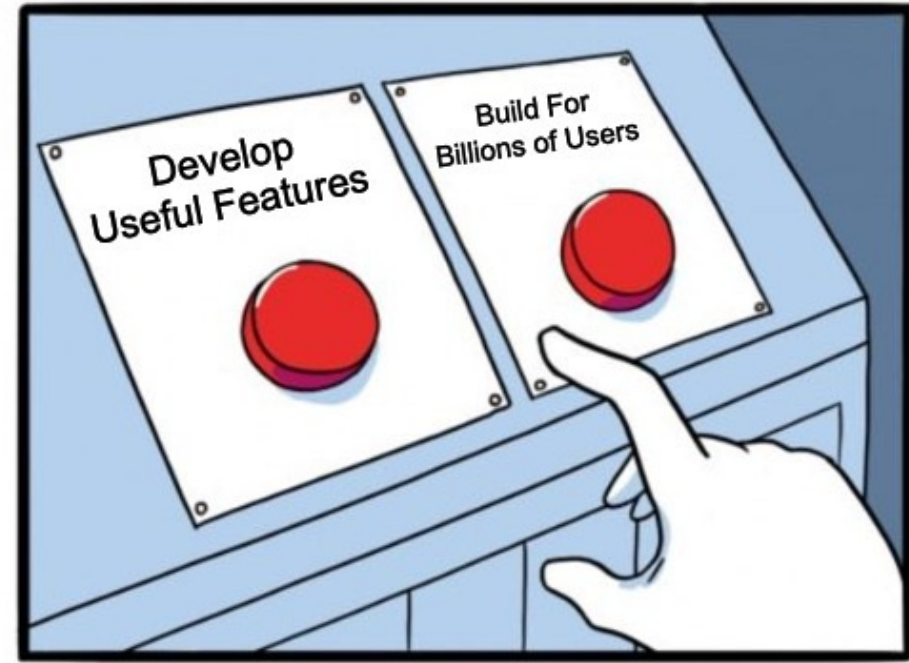


Scalability Rule of Thumb

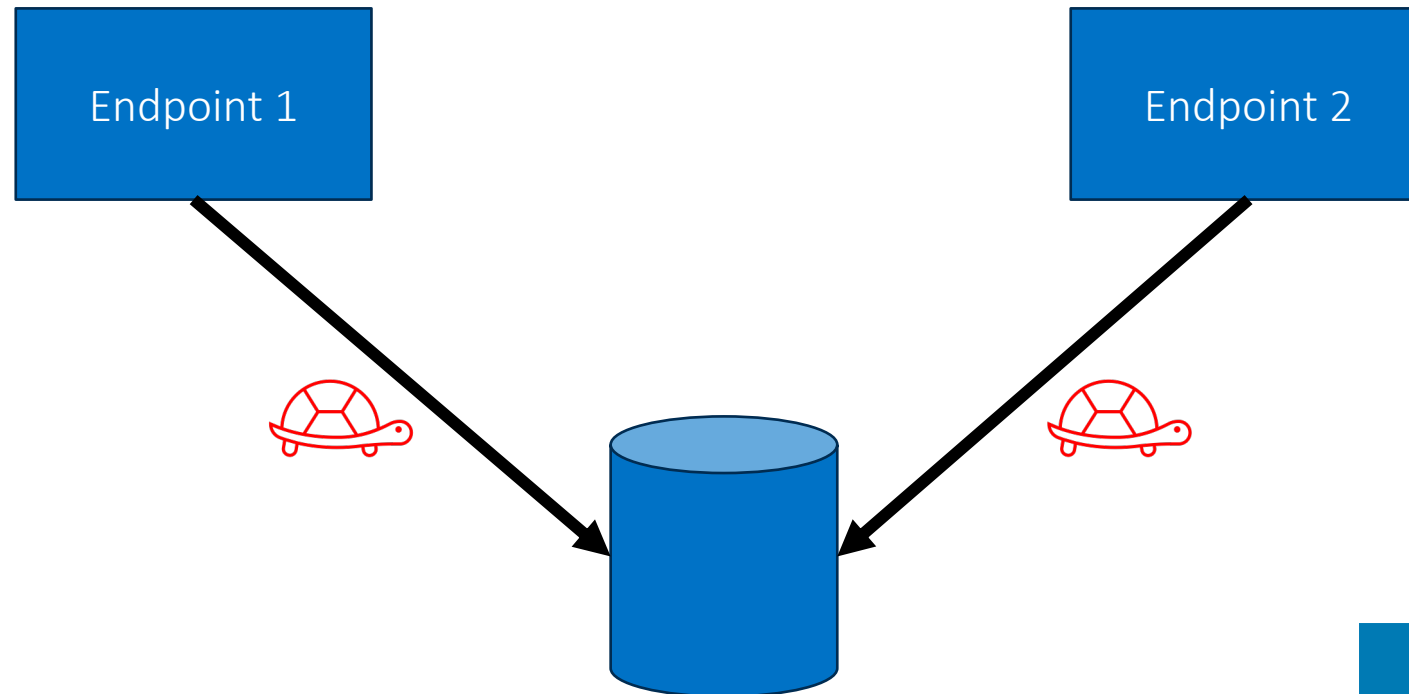


Premature Optimization

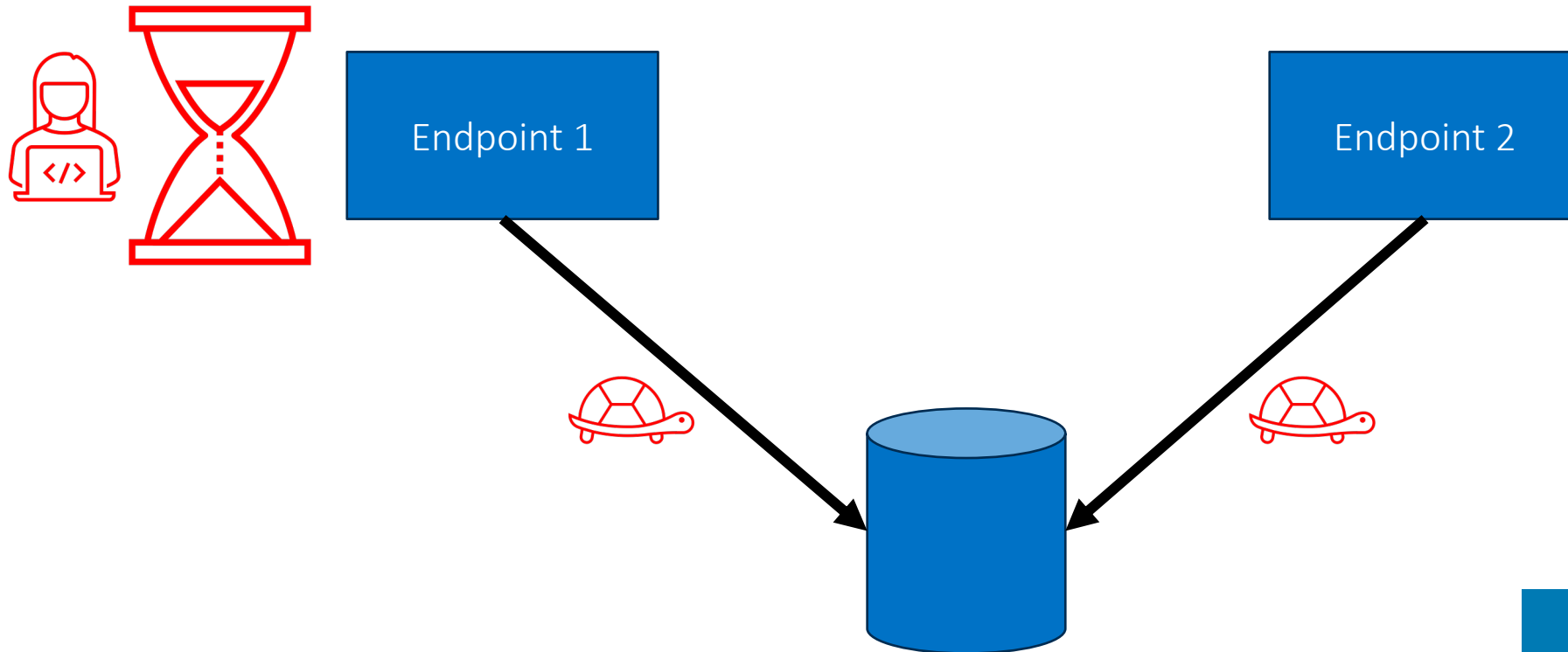
Over-Engineering Type 5



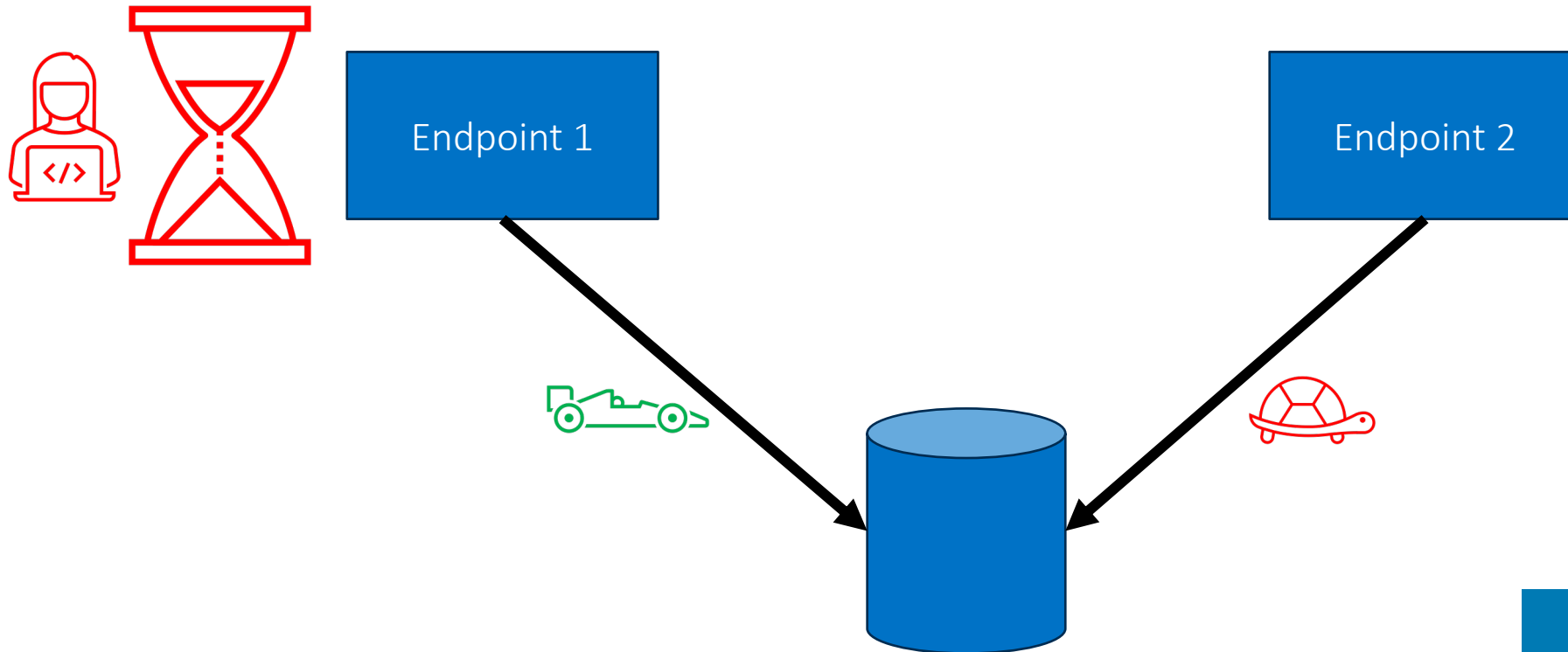
Premature Optimization



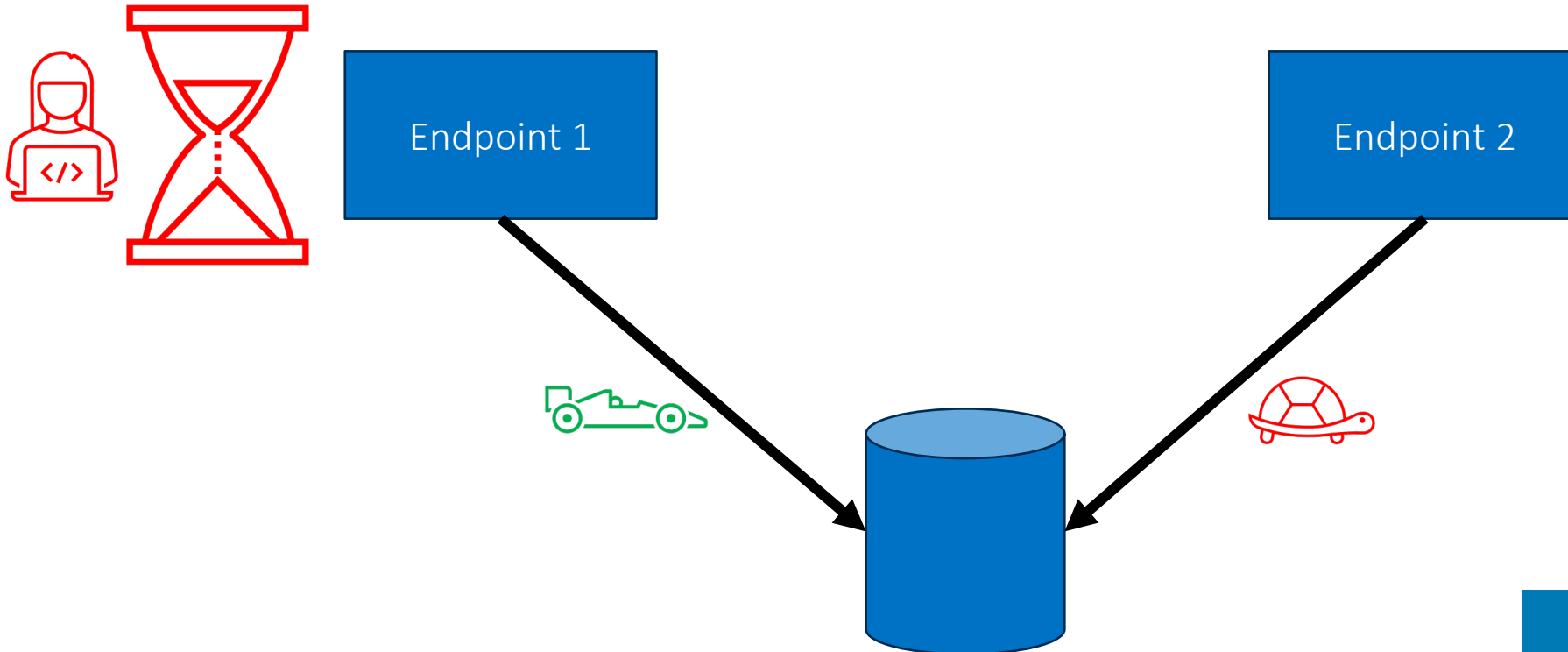
Premature Optimization



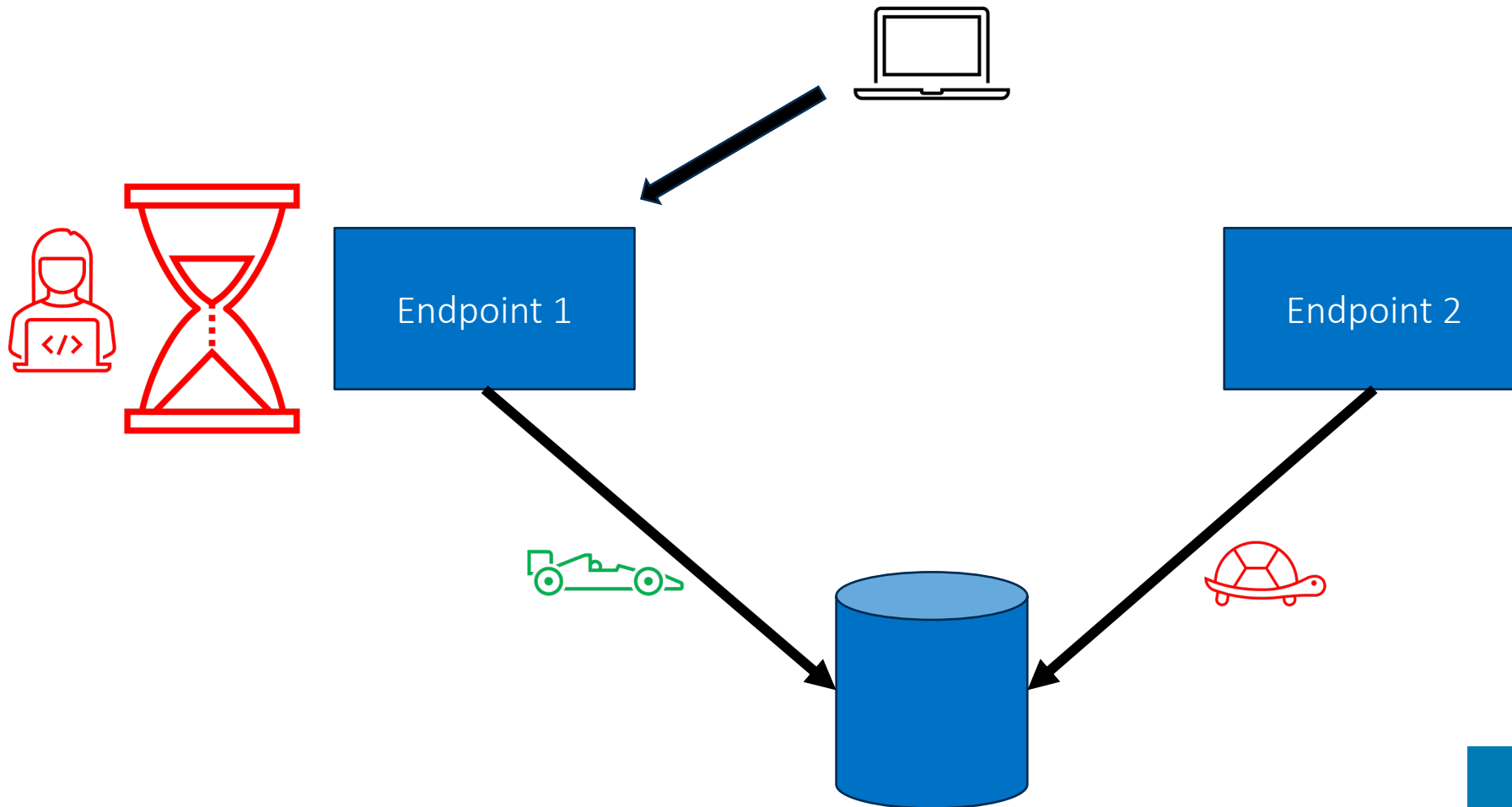
Premature Optimization



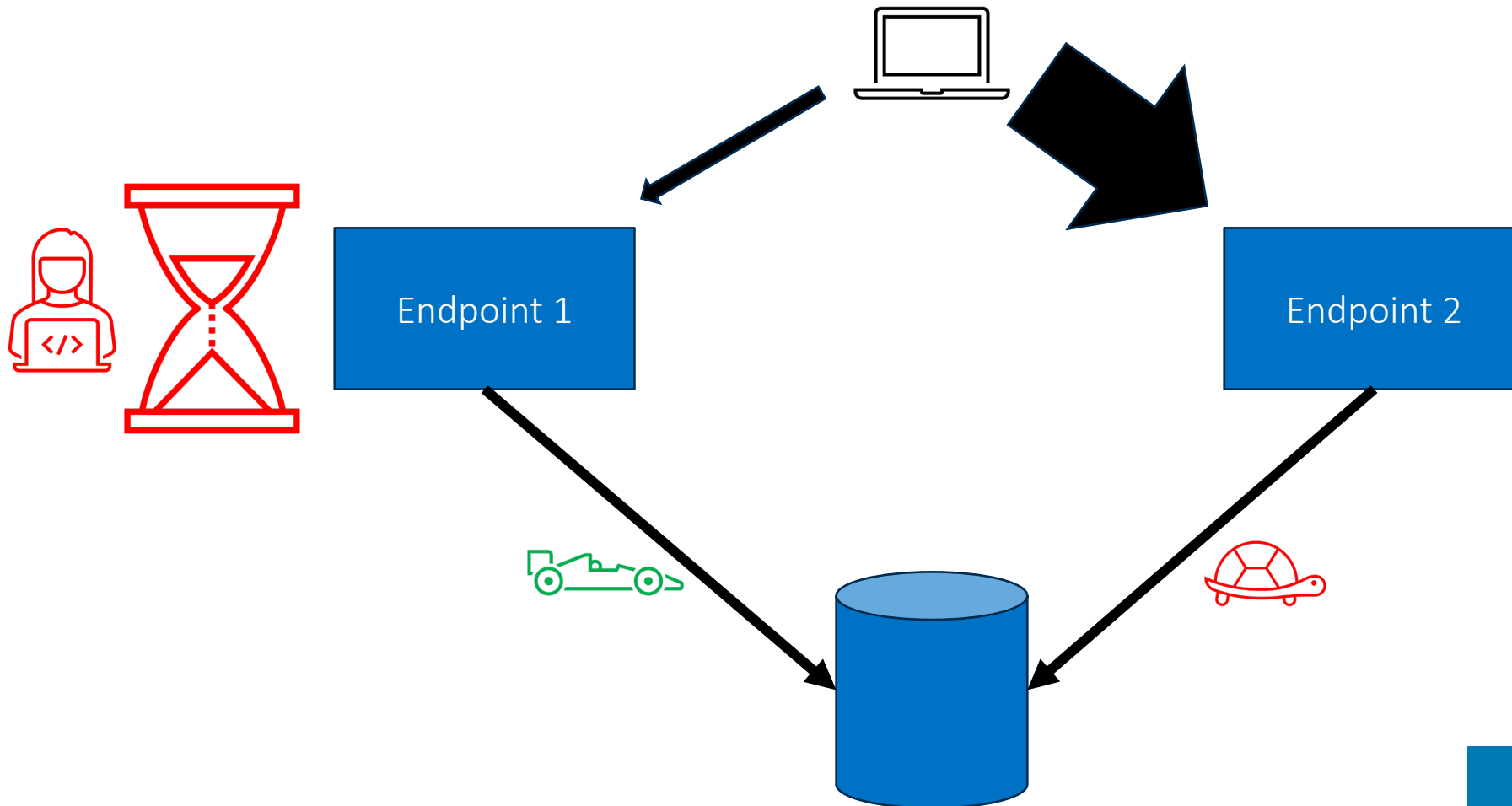
Premature Optimization



Premature Optimization



Premature Optimization



Overuse of Design Patterns

Over-Engineering Type 6



Design Patterns

Abstract Factory

Builder

Factory Method

Object Pool

Prototype

Singleton

Adapter

Bridge

Composite

Decorator

Facade

Flyweight

Private Class Data

Proxy

Chain of
responsibility

Command

Interpreter

Iterator

Mediator

Memento

Null Object

Observer

State

Strategy

Template method

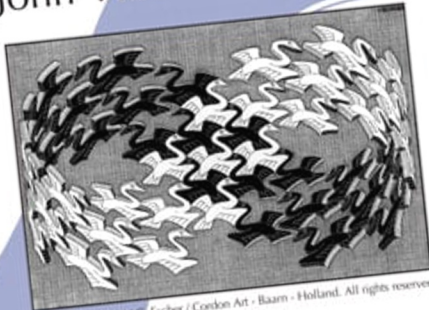
Visitor



Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides

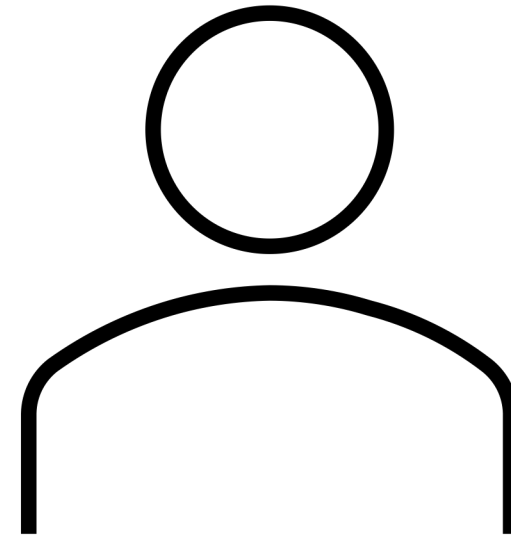


Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

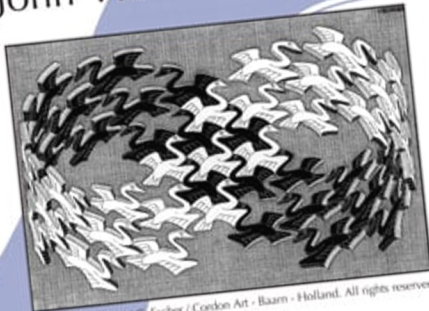


TRAILHEAD
TECHNOLOGY PARTNERS

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides

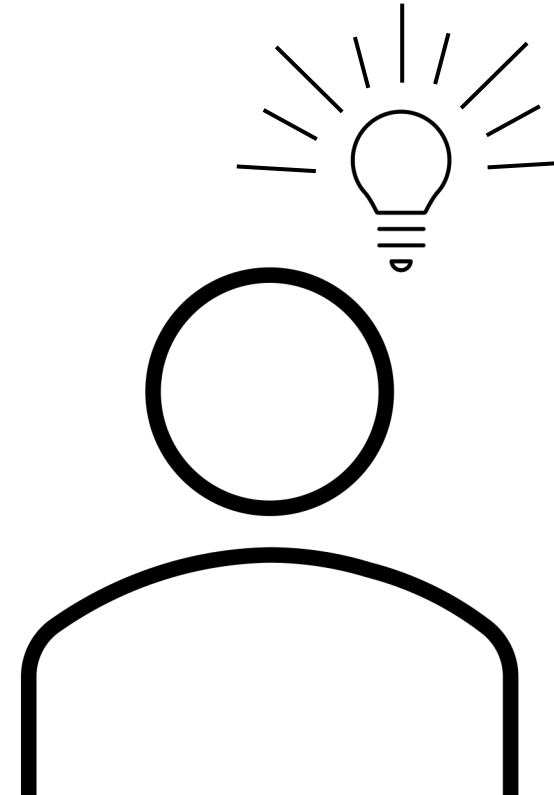


Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



TRAILHEAD
TECHNOLOGY PARTNERS

“If the only
tool **you have is**
a hammer, it is
tempting to
treat **everything**
as if it were a
nail.”

- Abraham Maslow



TRAILHEAD
TECHNOLOGY PARTNERS

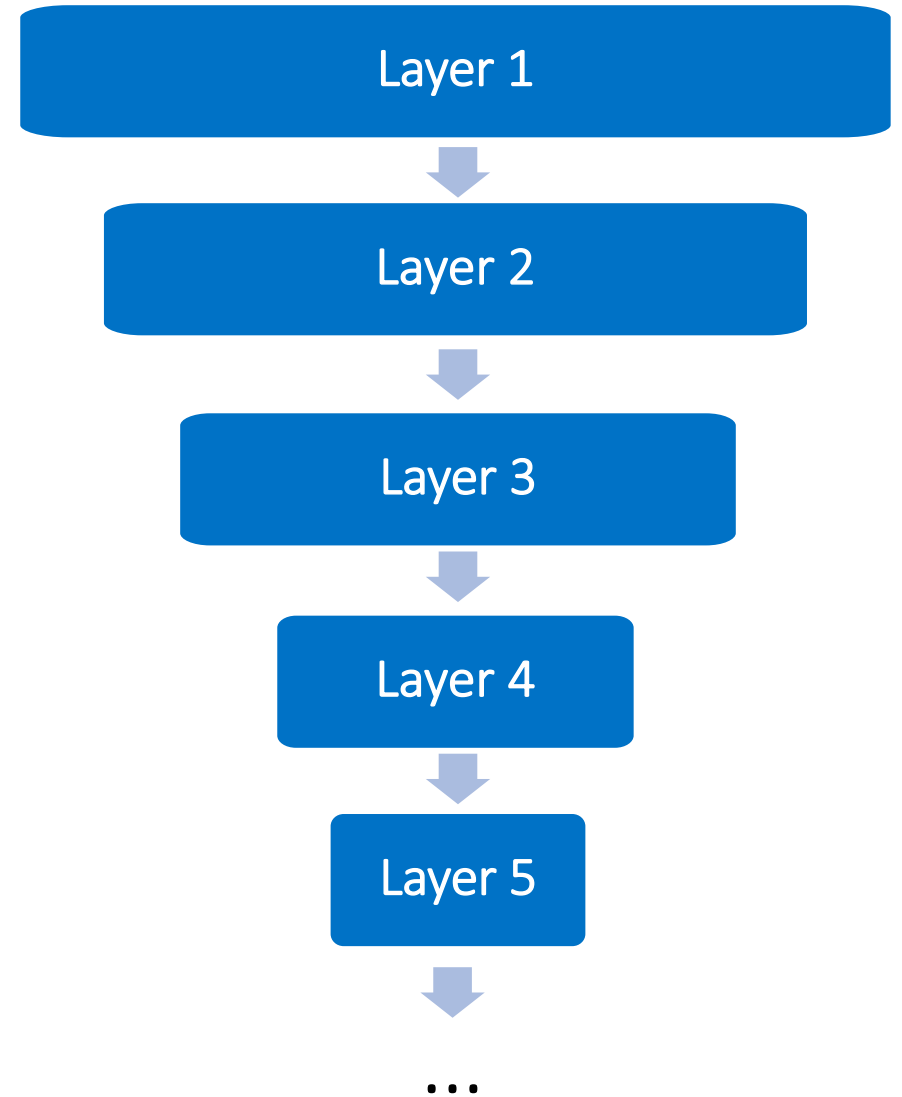
Lasagna Architecture

Over-Engineering Type 7



TRAILHEAD
TECHNOLOGY PARTNERS

Too Many Layers



Vertical Slice Architecture

Vertical slice architecture is a **software architecture** pattern that **organizes** code **by features** or use cases **instead of technical** concerns.

No more layers for their own sake!

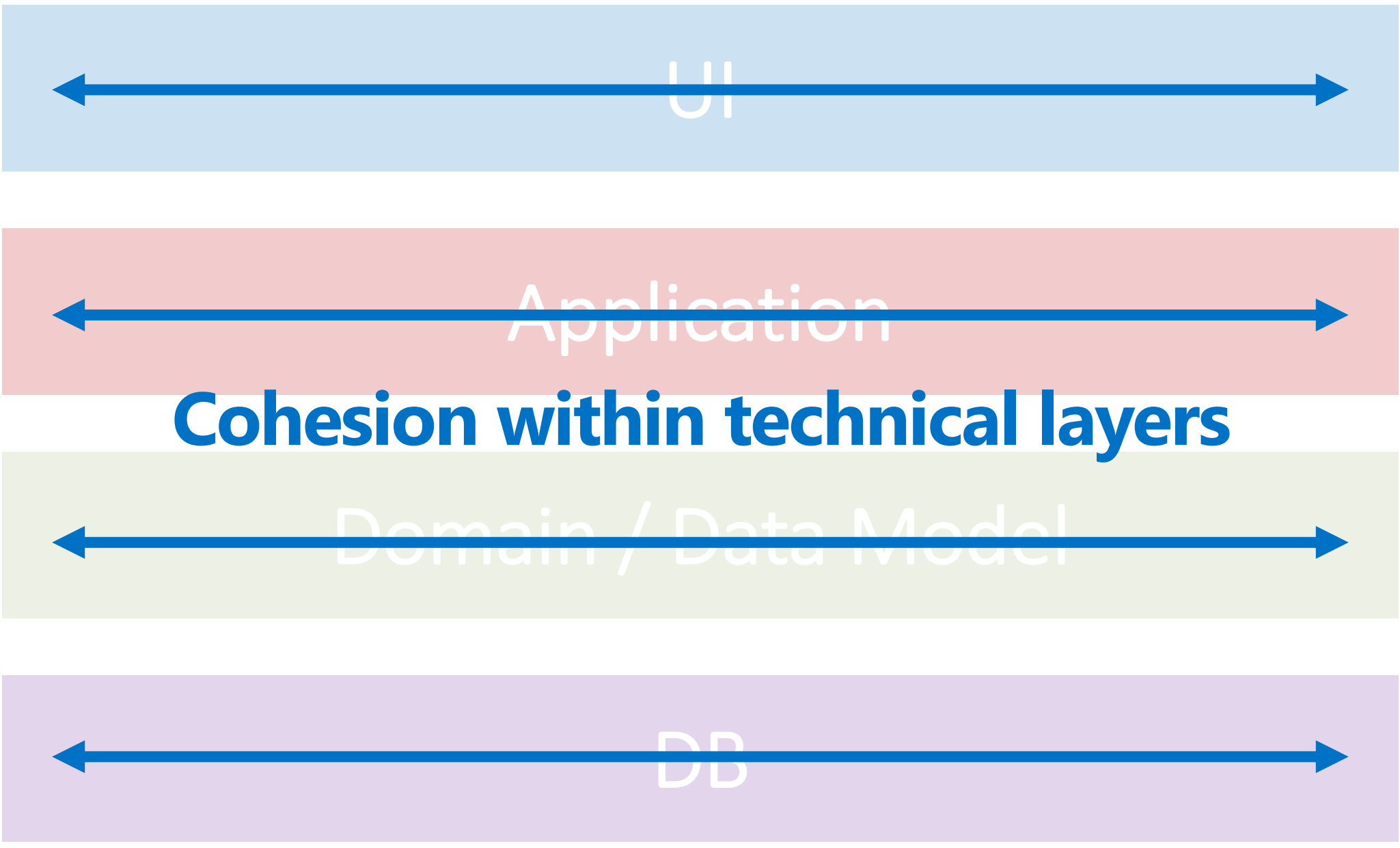


UI

Application

Domain / Data Model

DB



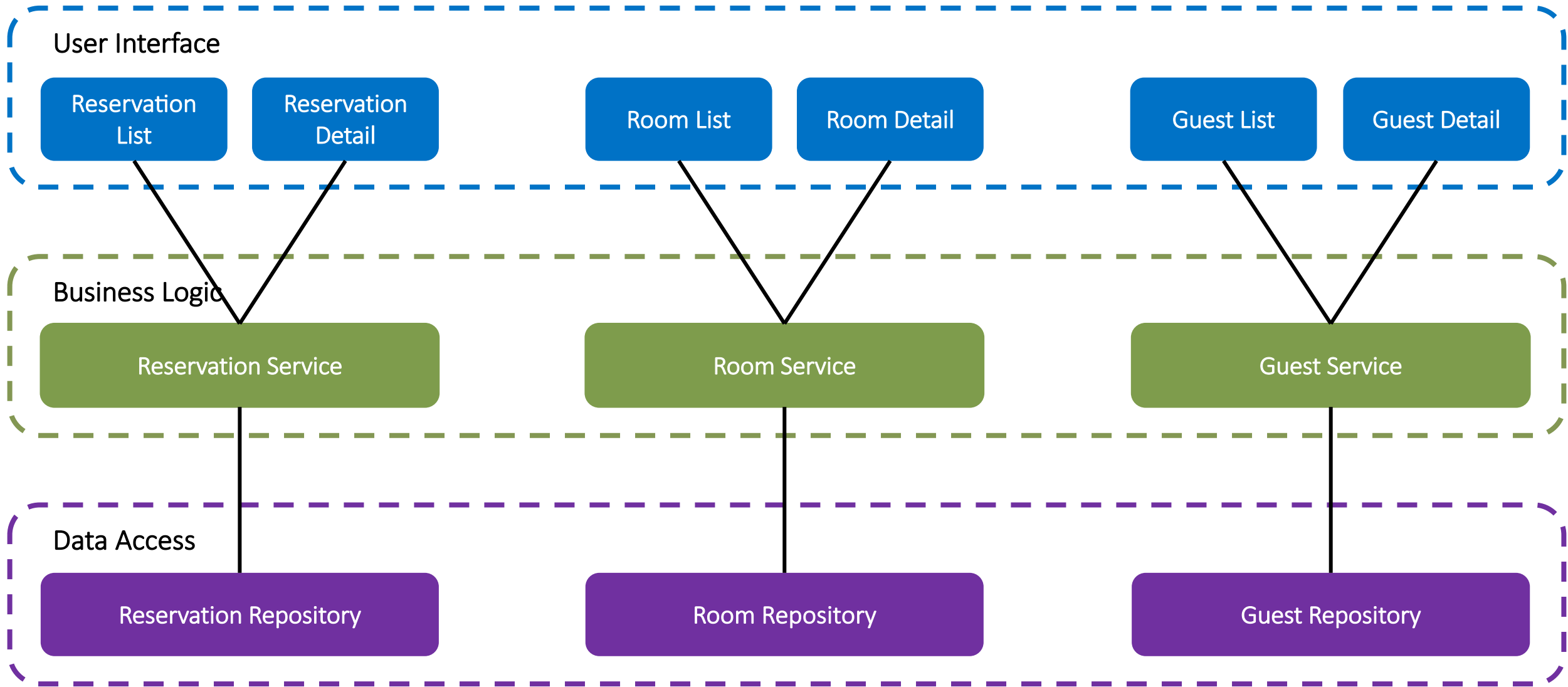
UI

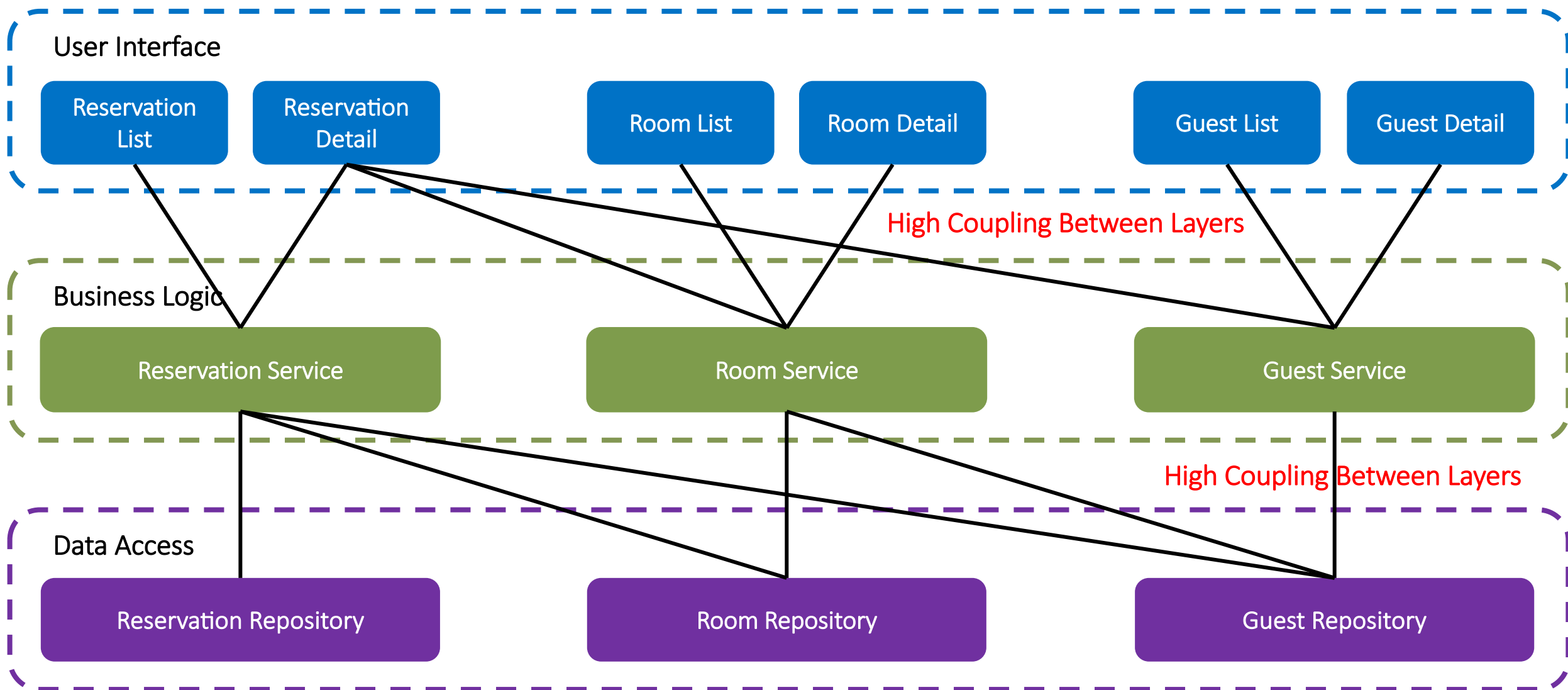
Application

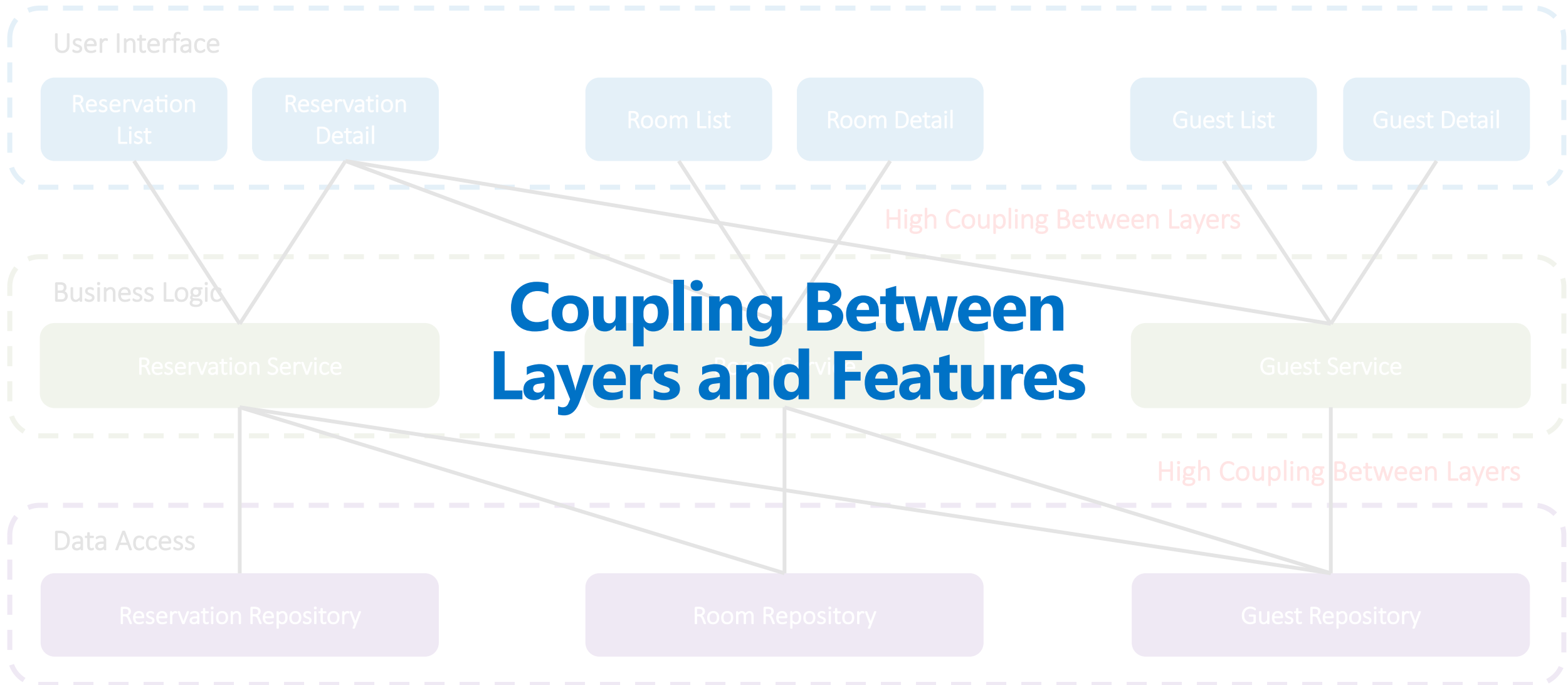
Cohesion within technical layers

Domain / Data Model

DB







UI

Application

Domain / Data Model

DB

UI

Application

Domain / Data Model

DB

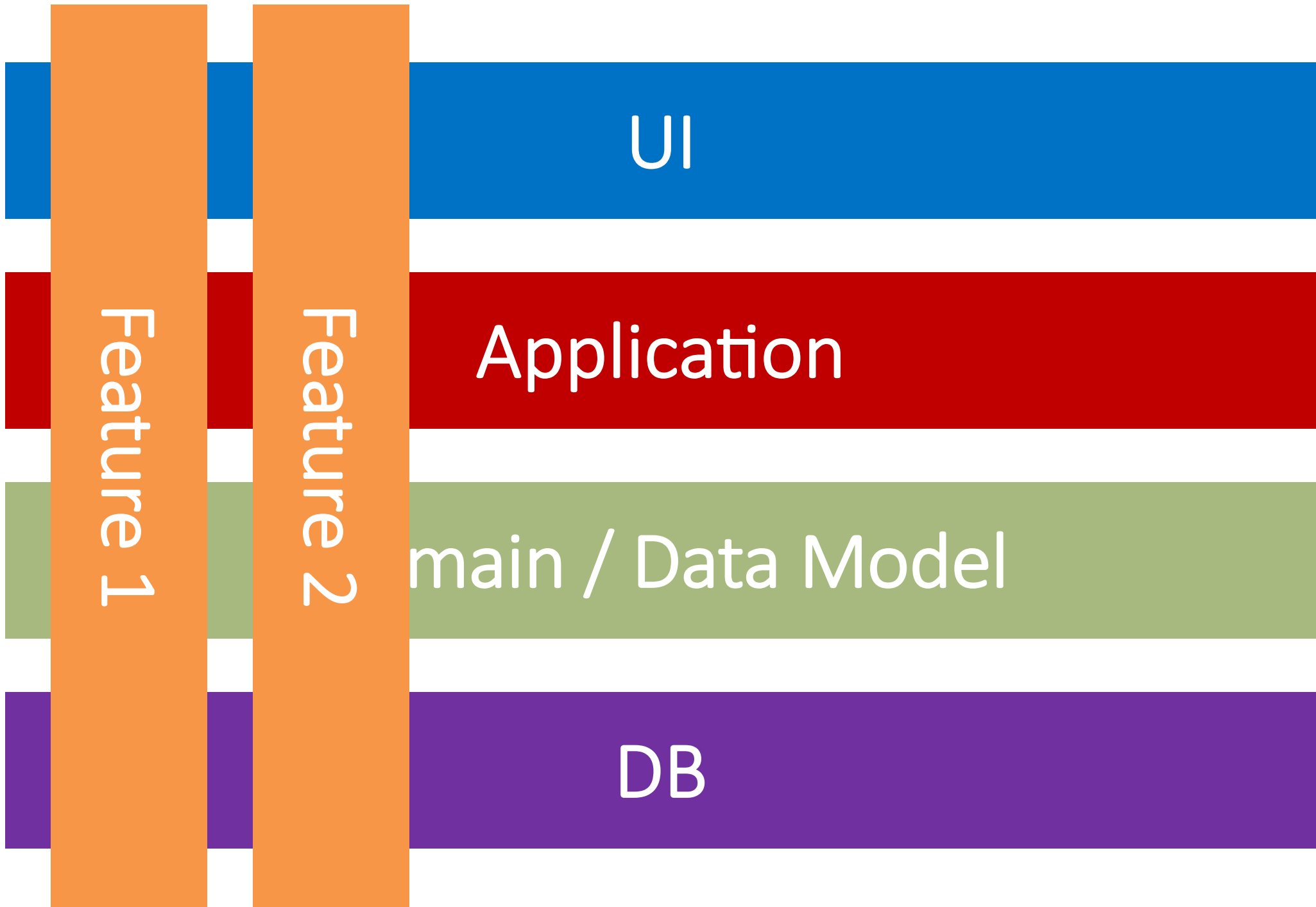
UI

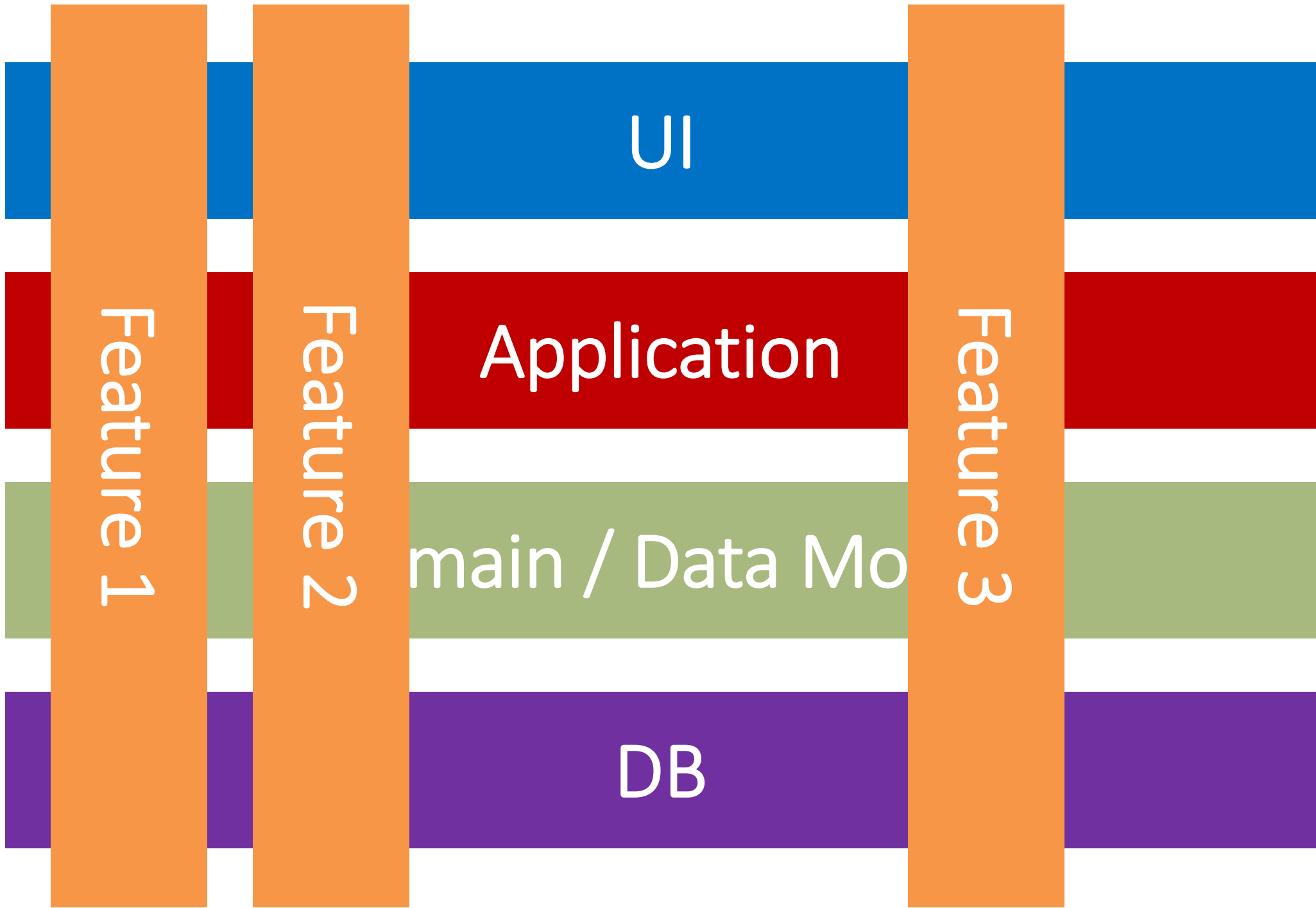
Application

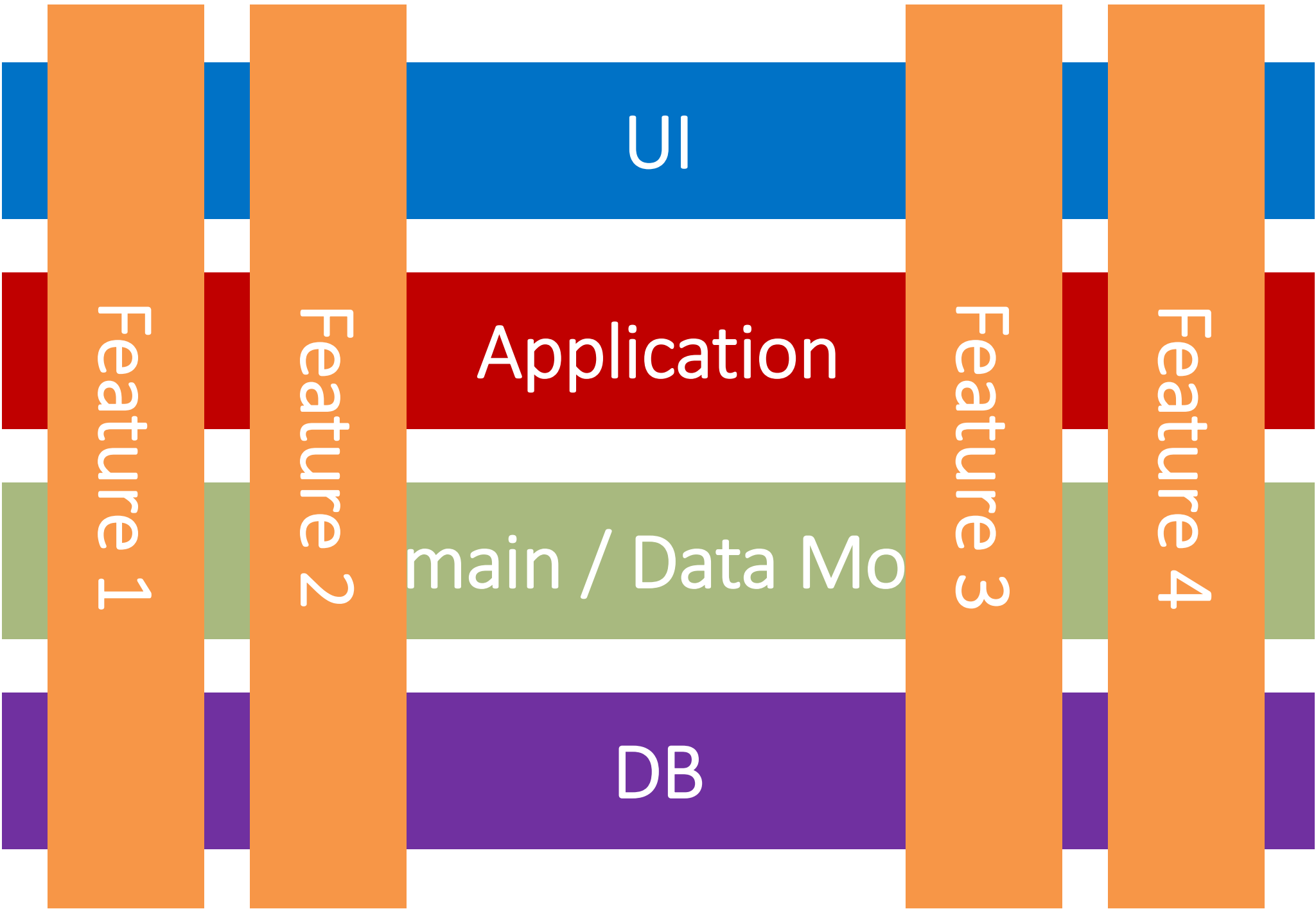
Domain / Data Model

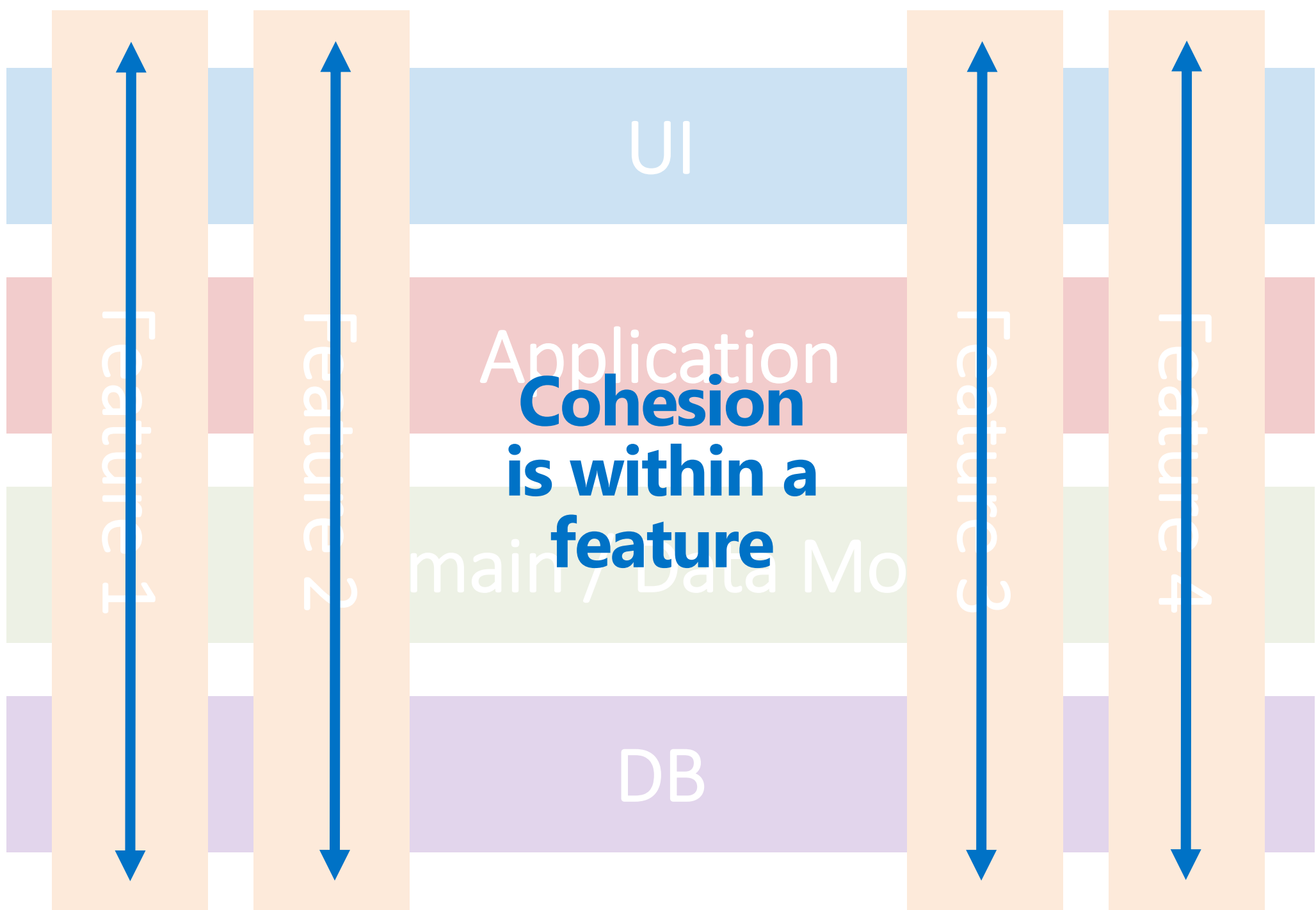
DB

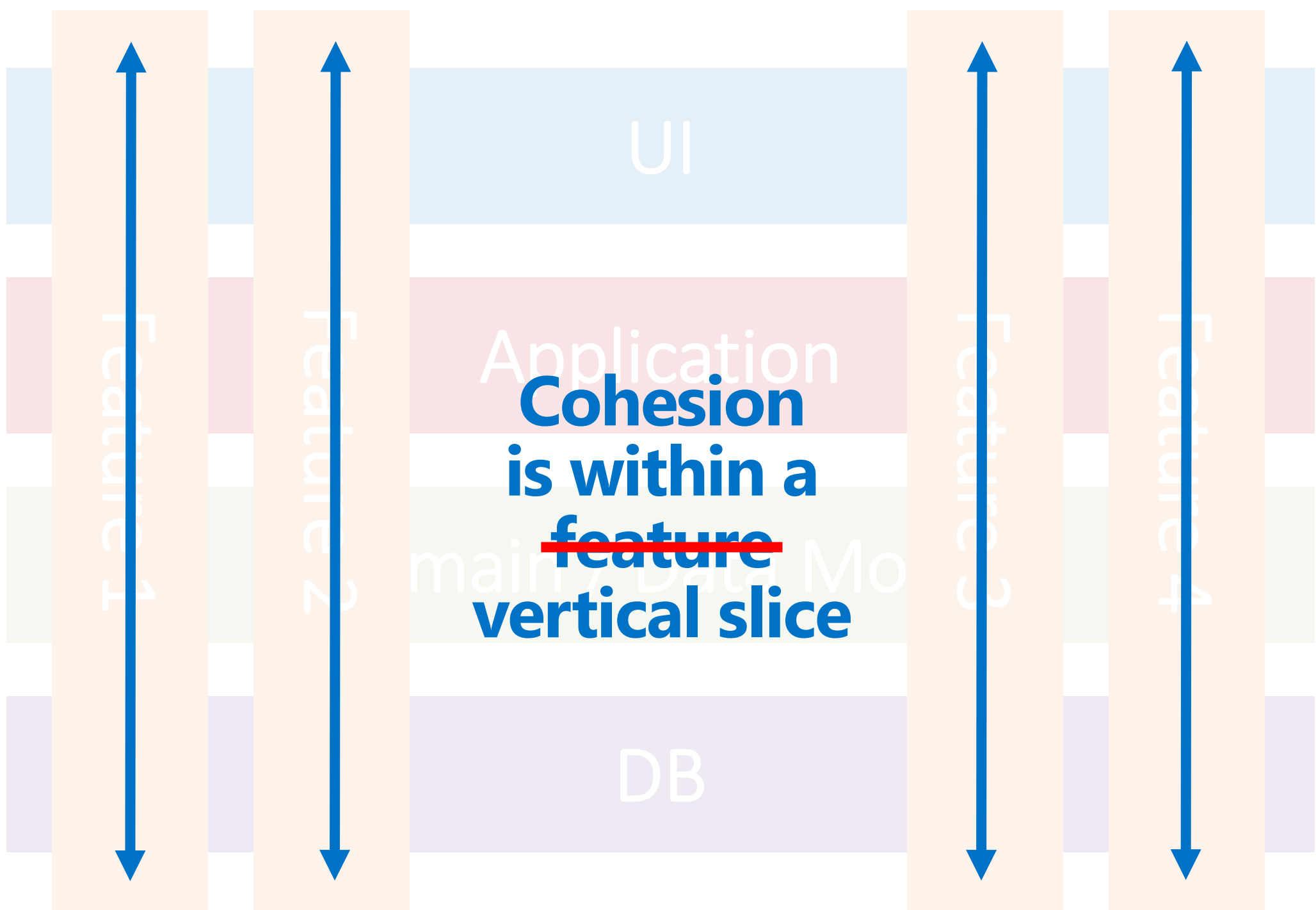
Feature 1



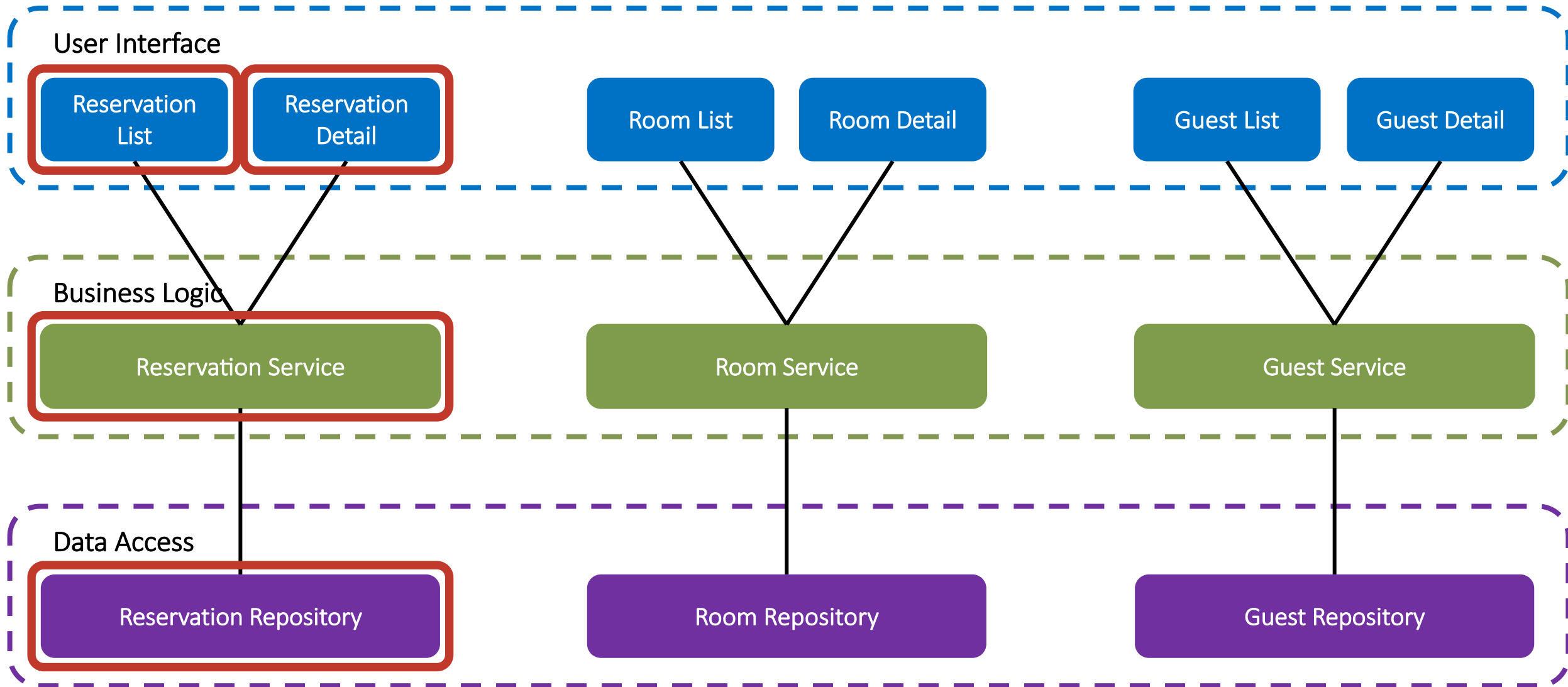


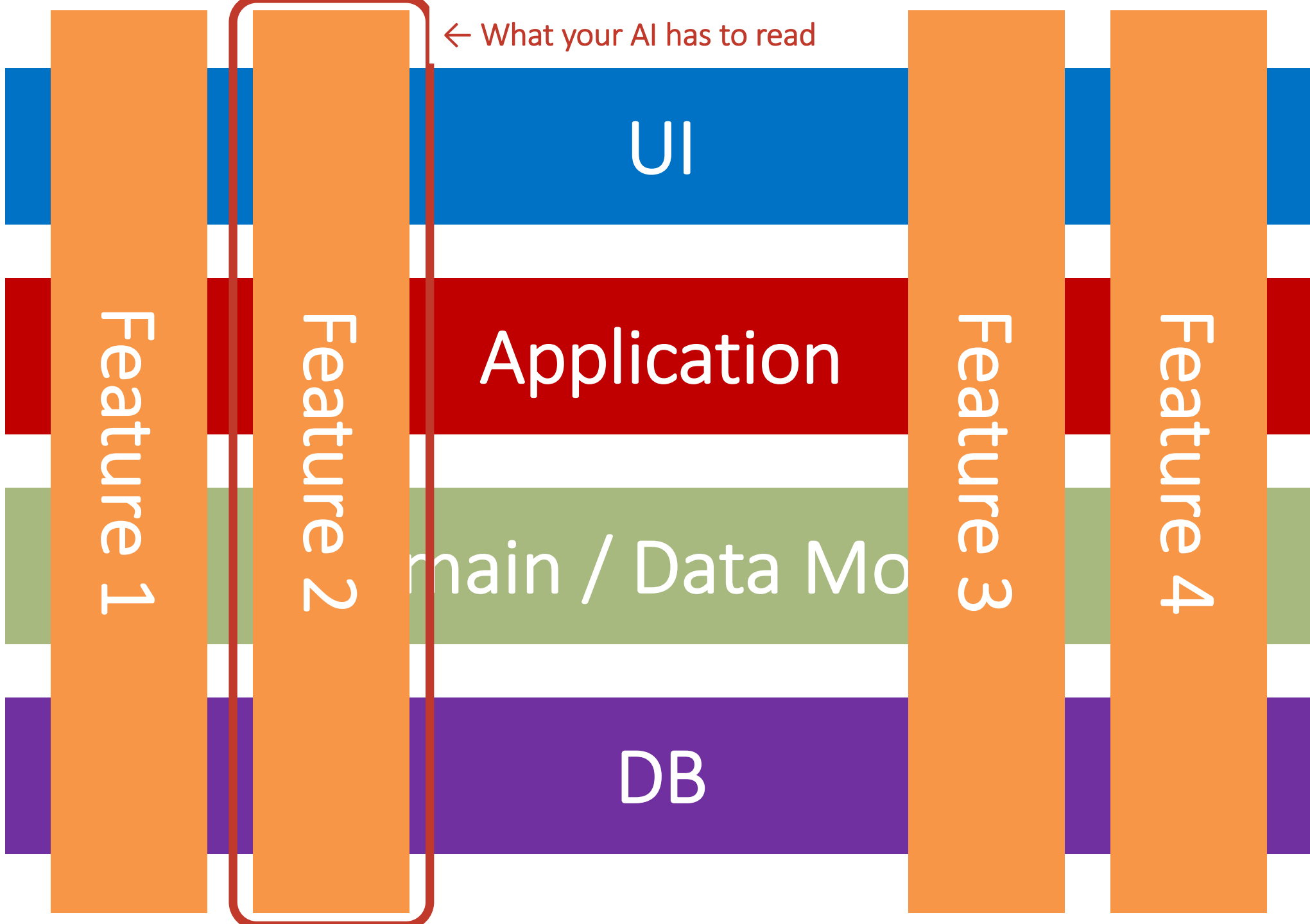






What your AI has to read to change reservations





← What your AI has to read

UI

Application

Main / Data Model

DB

Feature 1

Feature 2

Feature 3

Feature 4

Shiny Object Syndrome

Over-Engineering Type 8



Which Stack Is More Fun?

UI

Middle-Tier

Backend

Stuff You Already Know



UI



Middle-Tier



Backend

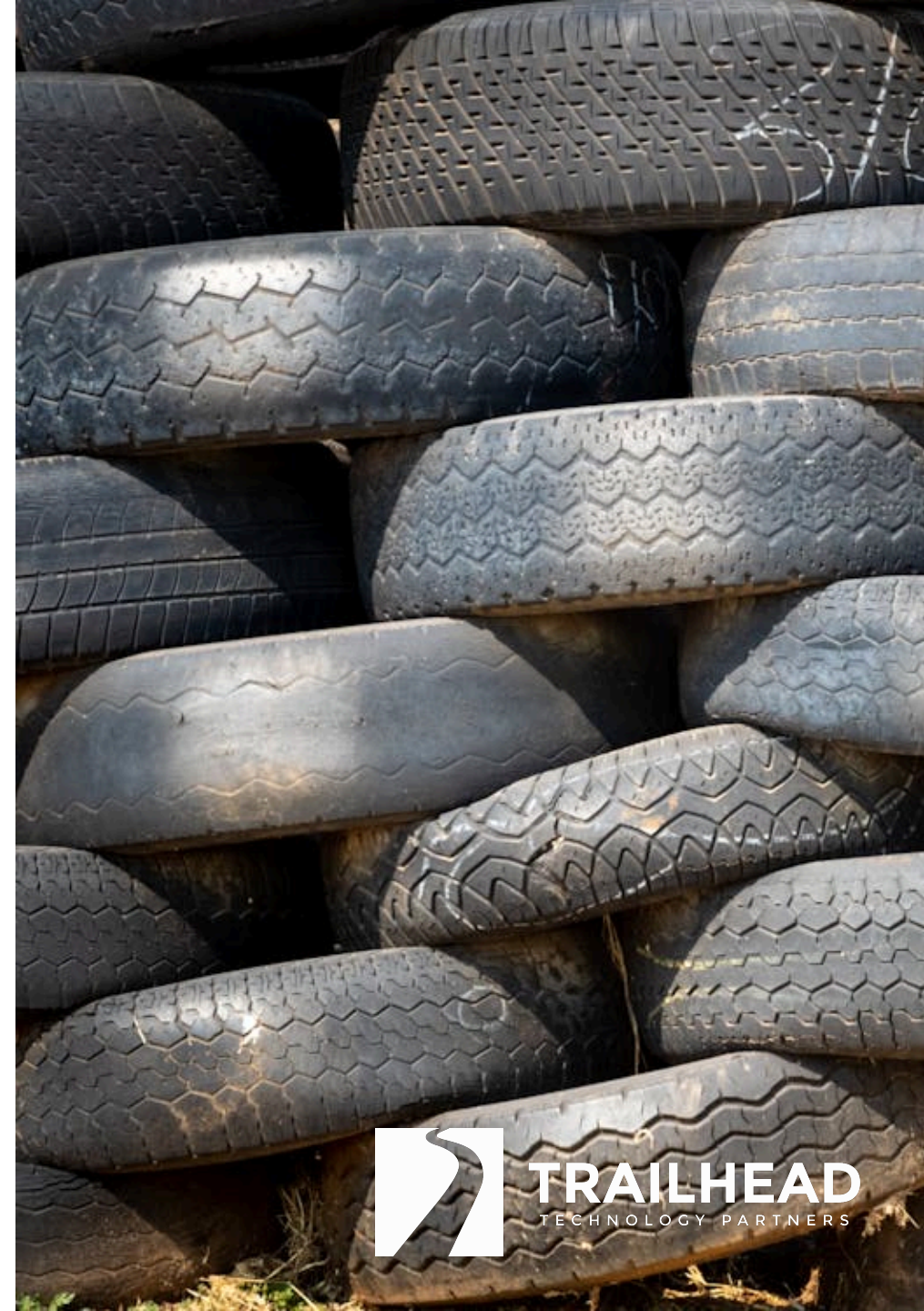
All New Stuff

Rolling Your Own

Over-Engineering Type 9



TRAILHEAD
TECHNOLOGY PARTNERS



Issues with Rolling Your Own



Increased Costs



Extended
Development Time



New Dev Training



Reduced Testing
Exposure

Issues with Rolling Your Own



Increased Costs



Extended
Development Time



New Dev Training



Reduced Testing
Exposure

AI Caveat*

- For certain dependencies building your own with AI is good:
 - ORM/mapping
 - Simple in-process mediators
 - A thin HTTP instead of a giant vendor SDK
 - Tiny utilities like slugify, string and date helpers
- Still not ideal
 - Frameworks, runtimes, HTTP servers
 - Event queues and message brokers
 - Crypto, auth, and identity
 - Retries and resilience

Misapplied Libraries/Frameworks

Over-Engineering Type 10



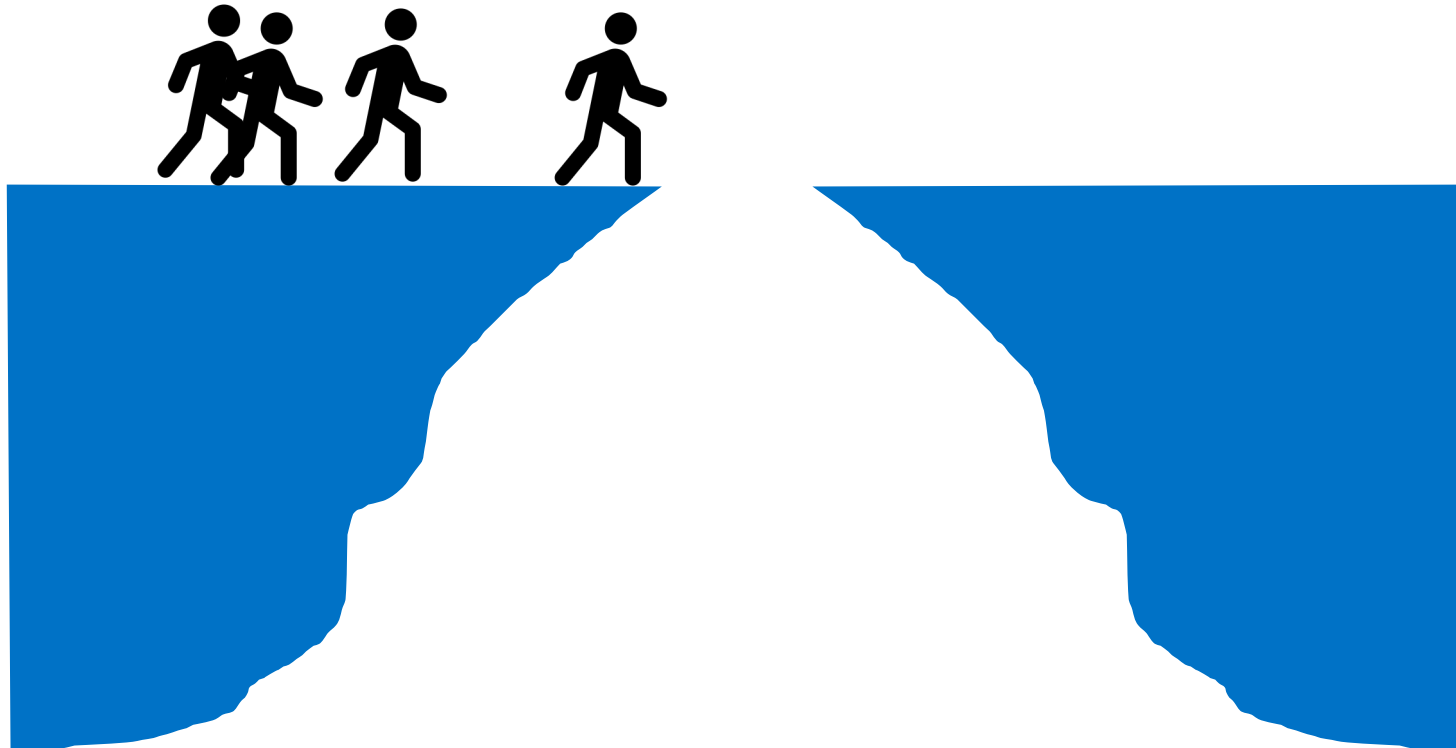
TRAILHEAD
TECHNOLOGY PARTNERS

10 **Rules** To Avoid Over-Engineering

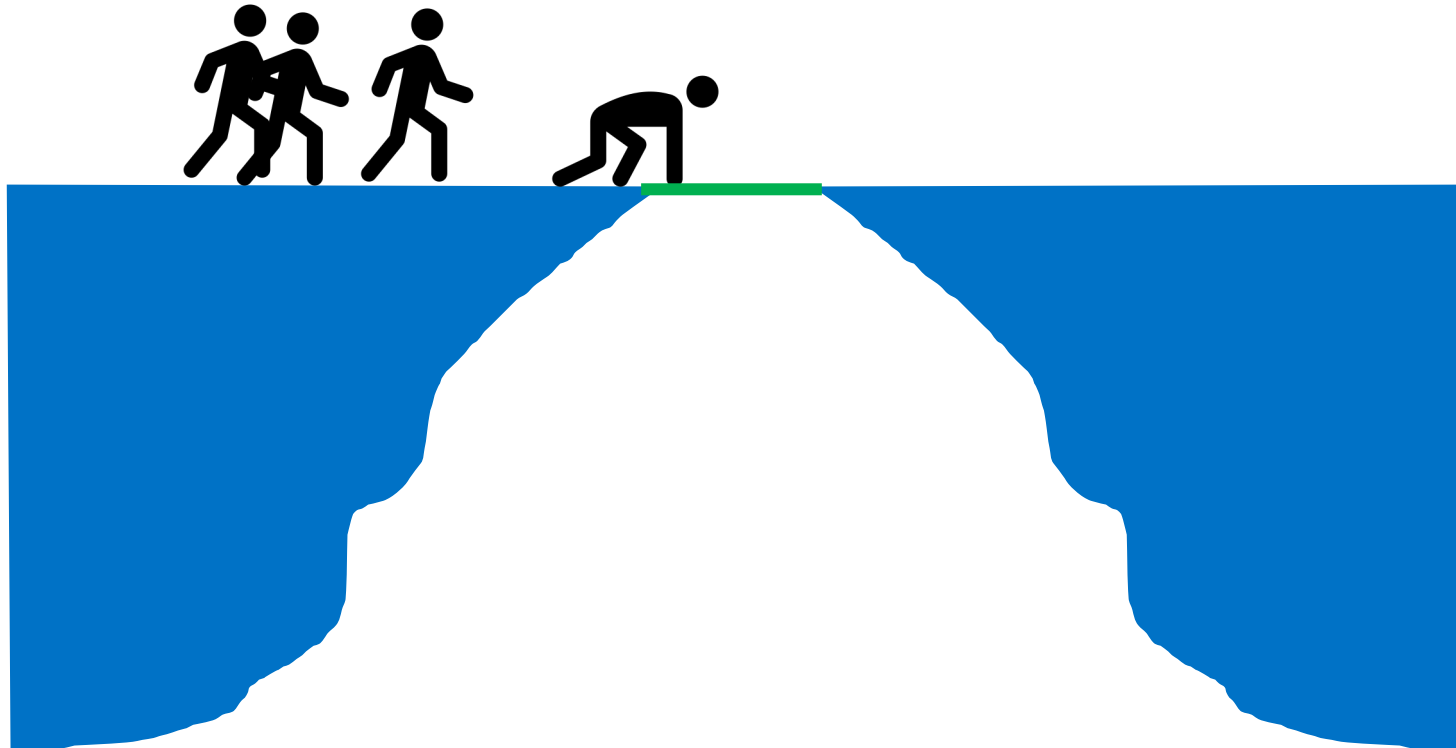
Document Your Engineering Decisions

Rule 1

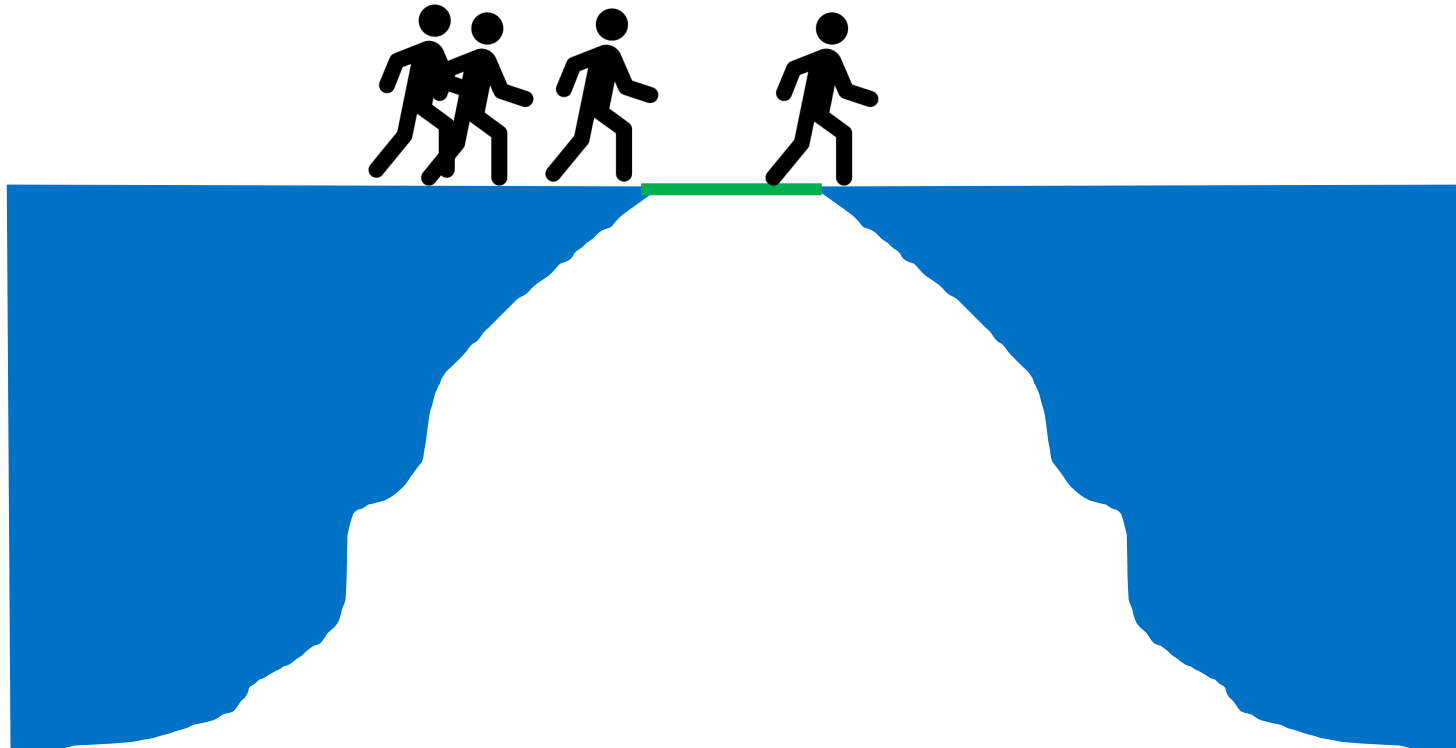
Avoid Mistakes In The Future



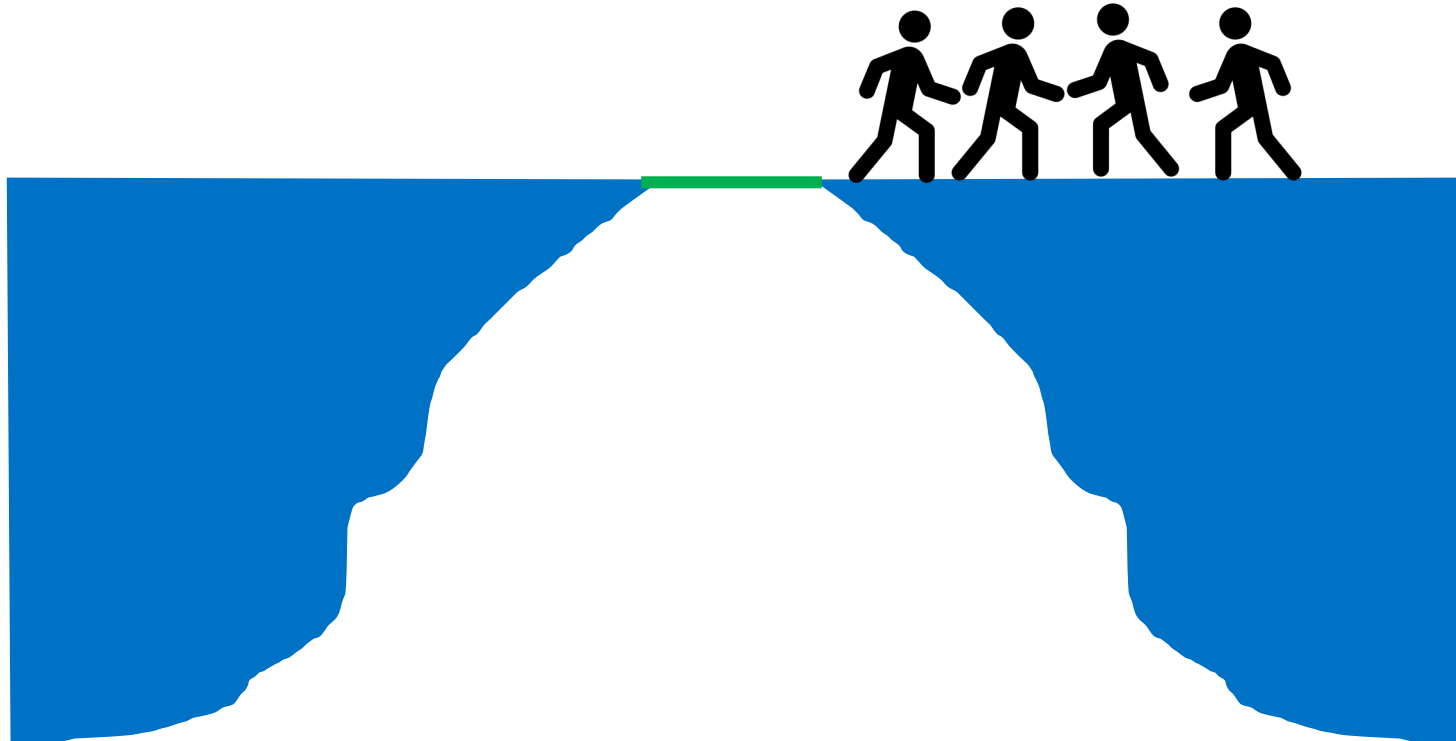
Avoid Mistakes In The Future



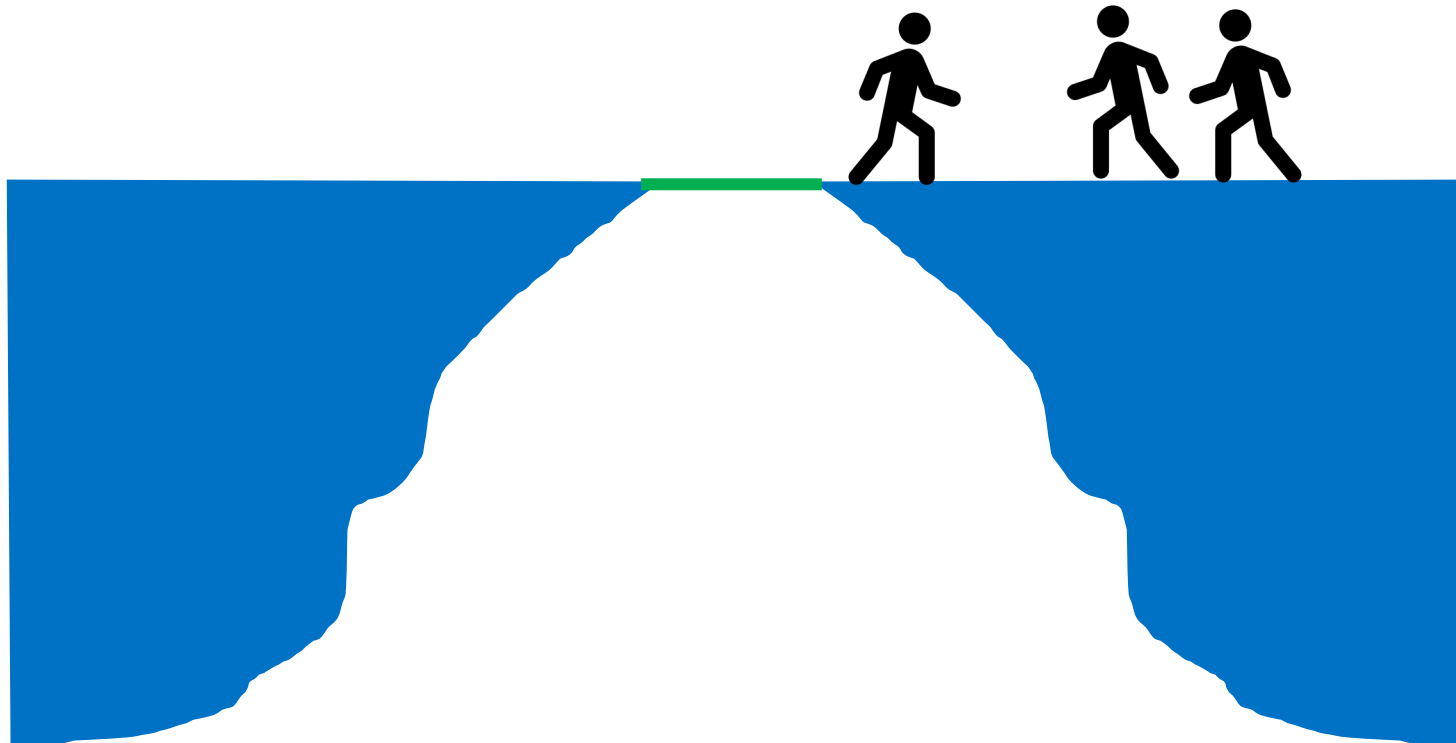
Avoid Mistakes In The Future



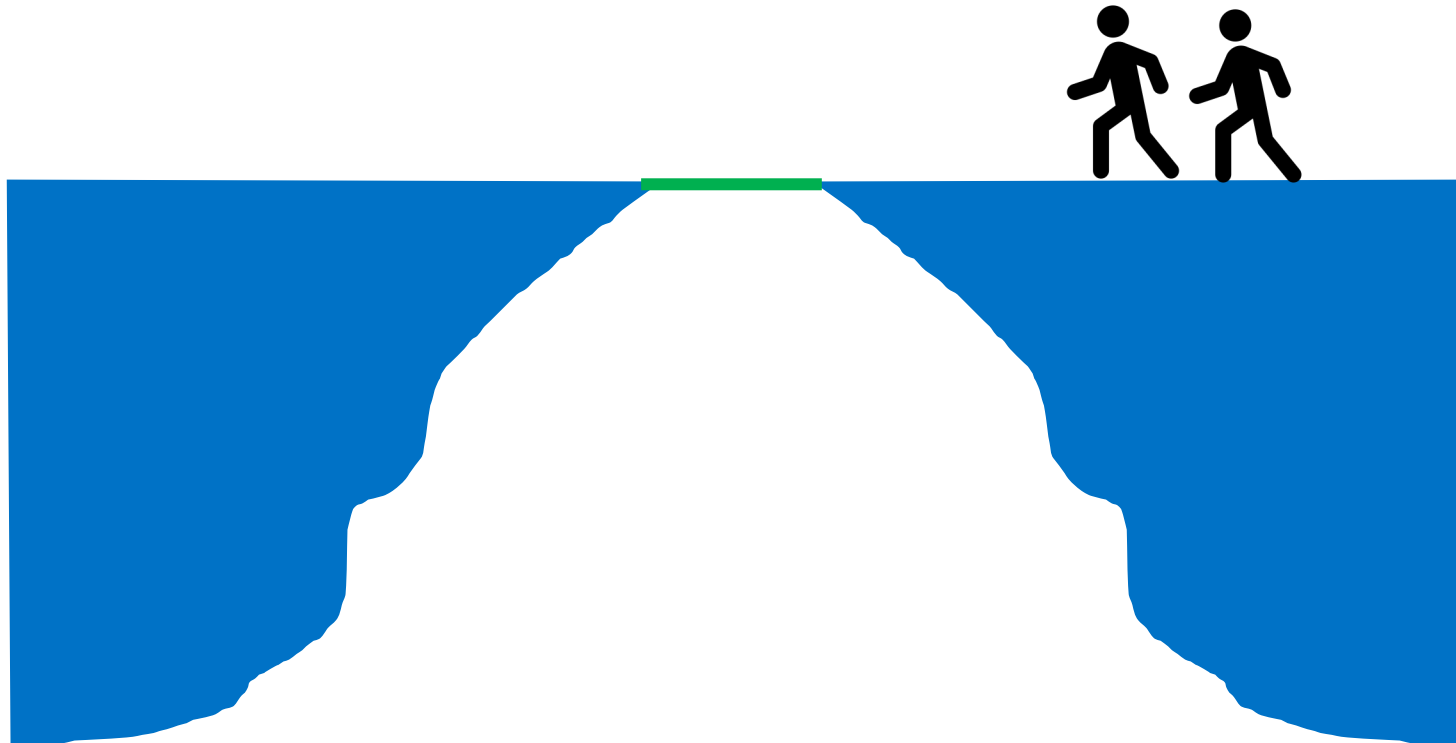
Avoid Mistakes In The Future



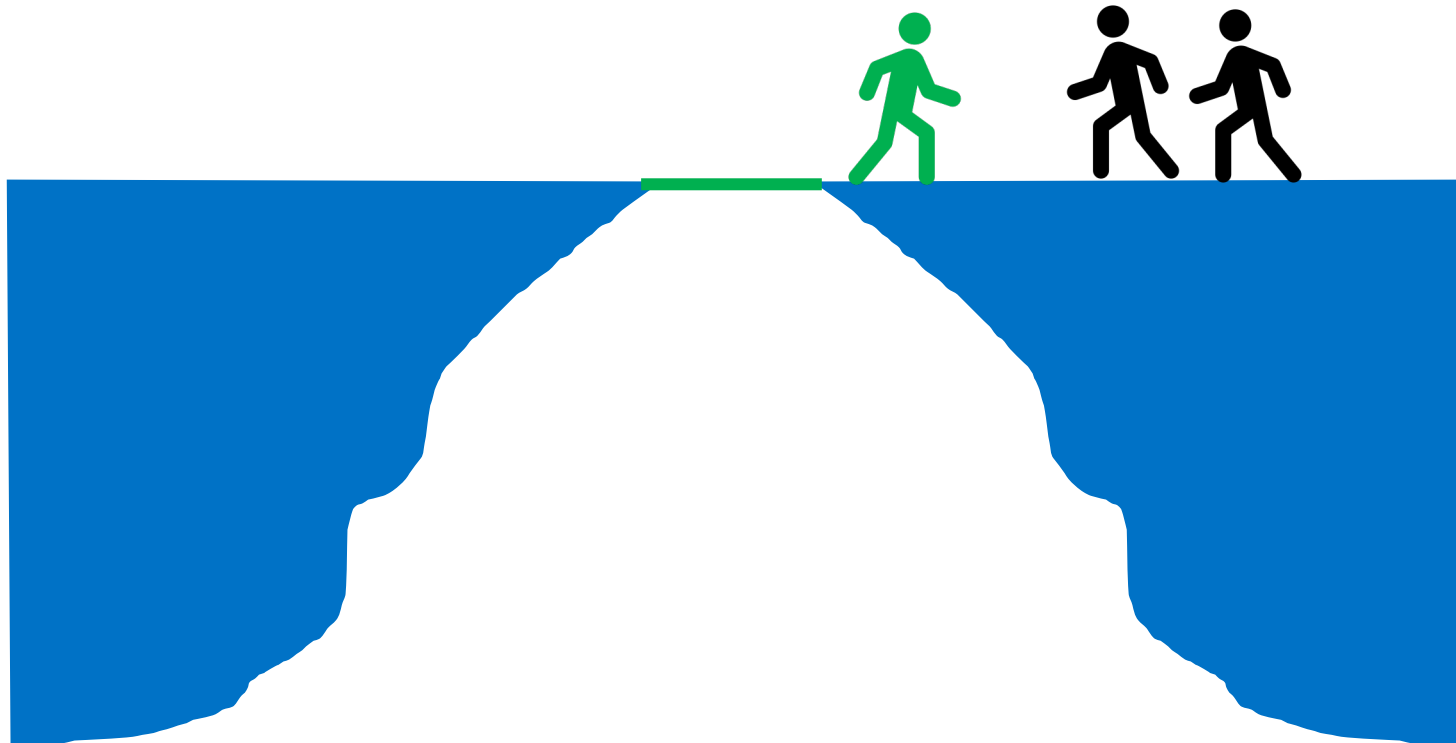
Avoid Mistakes In The Future



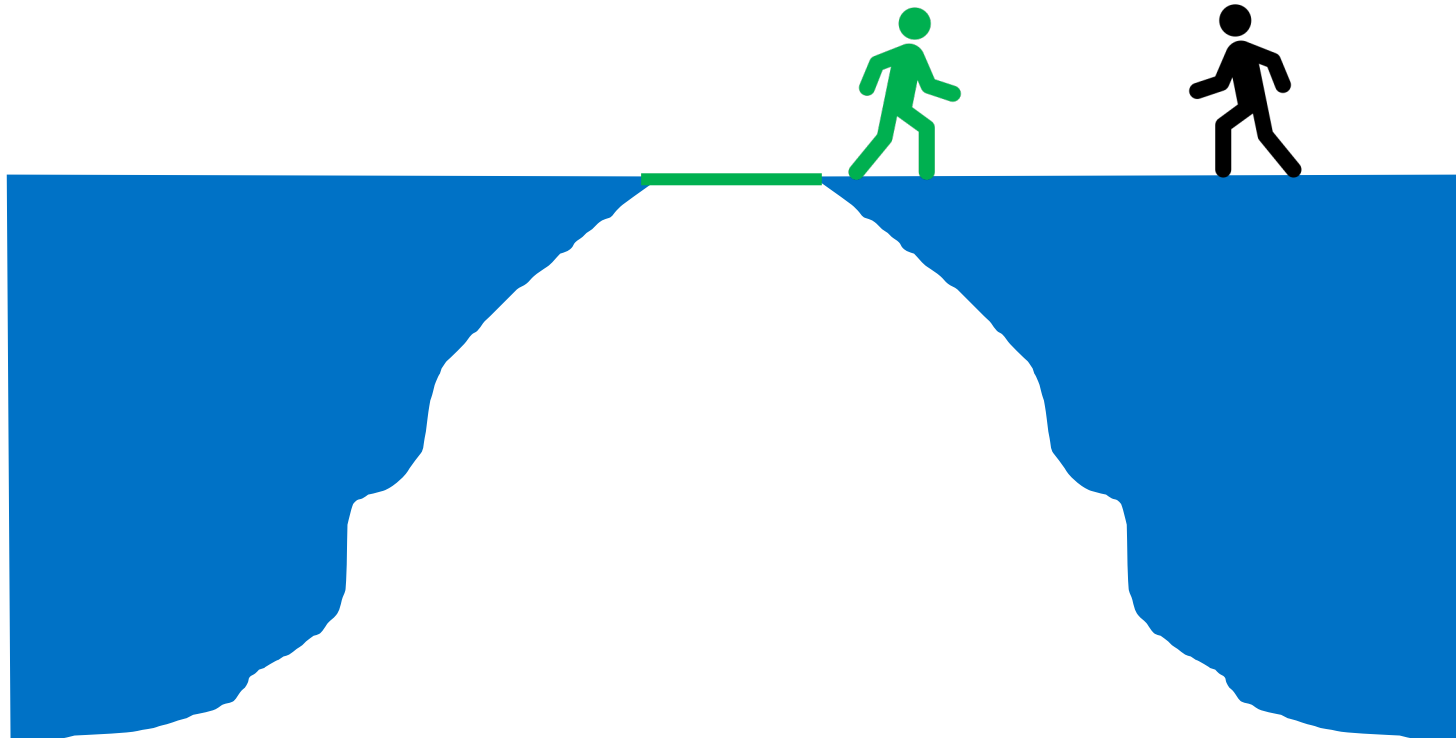
Avoid Mistakes In The Future



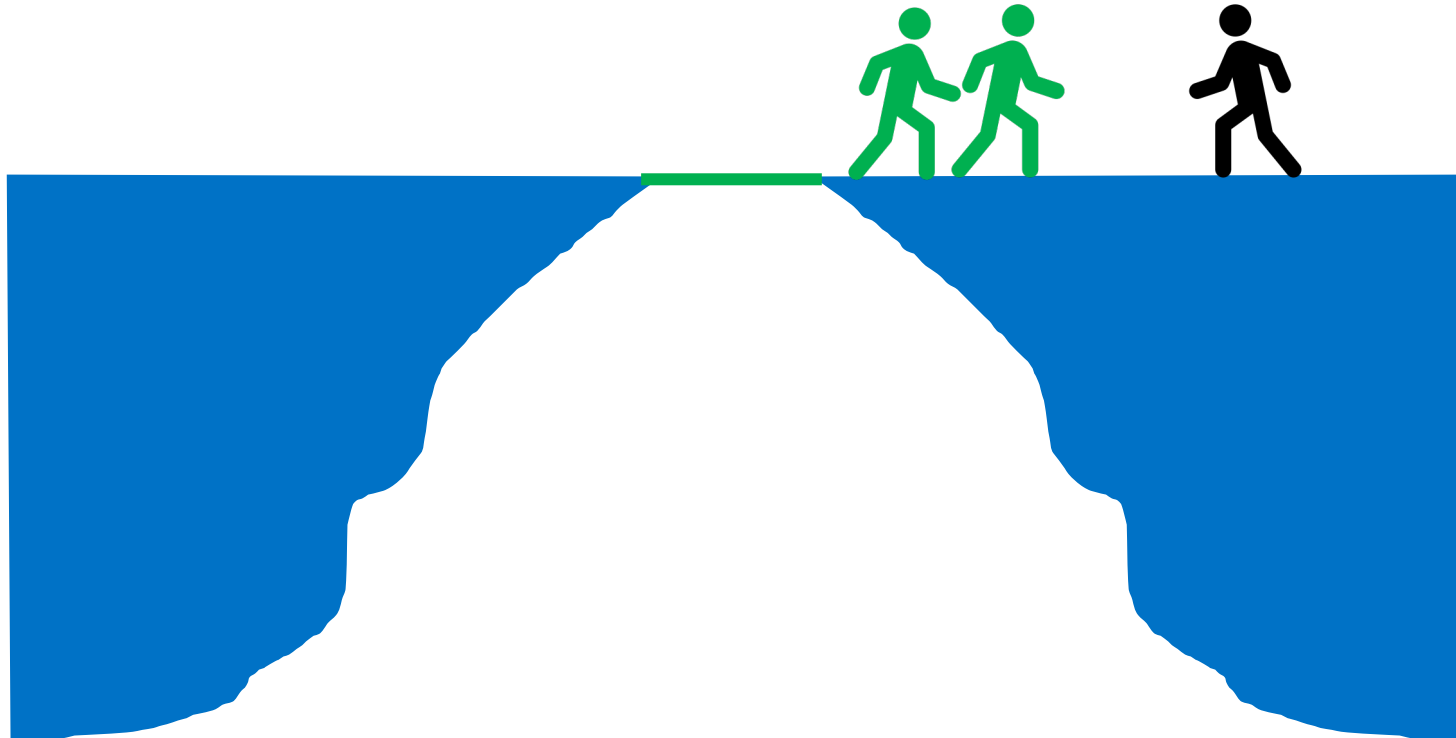
Avoid Mistakes In The Future



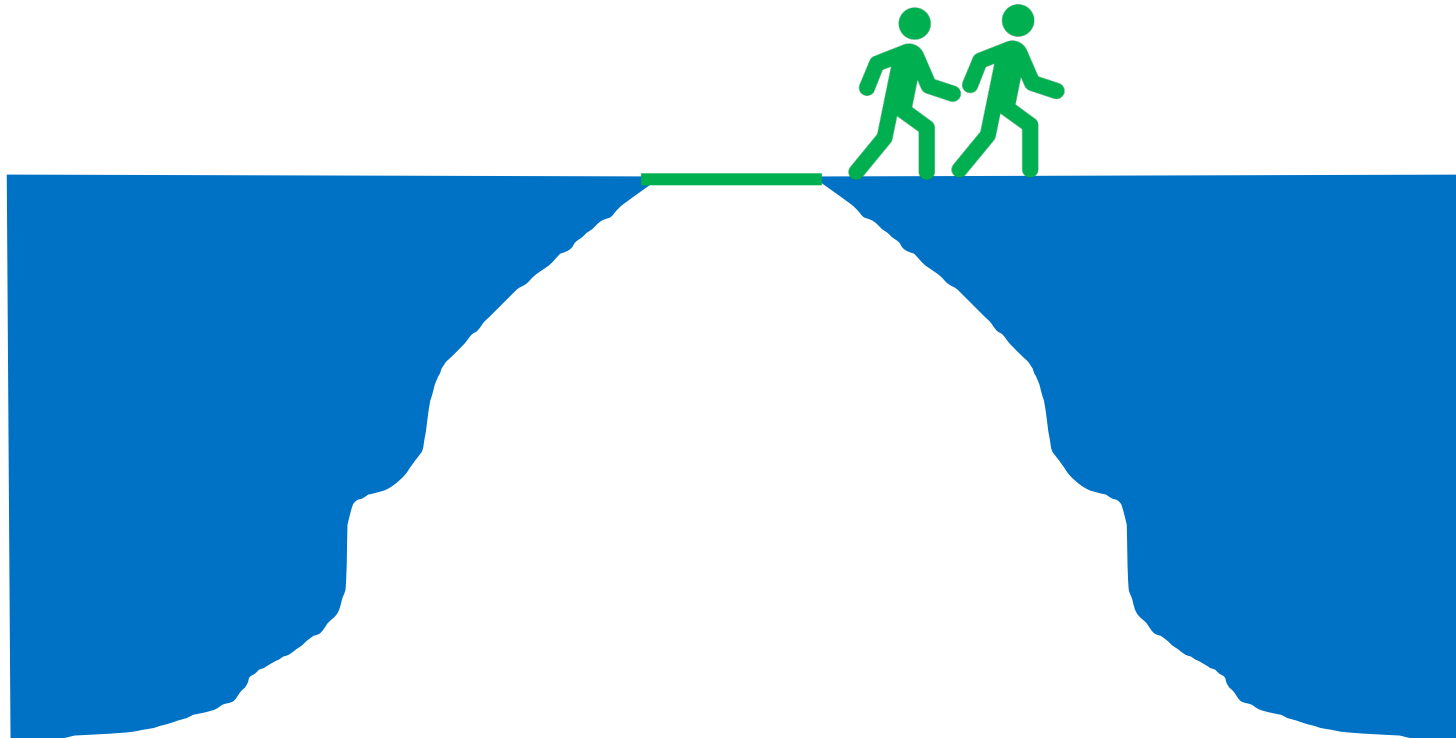
Avoid Mistakes In The Future



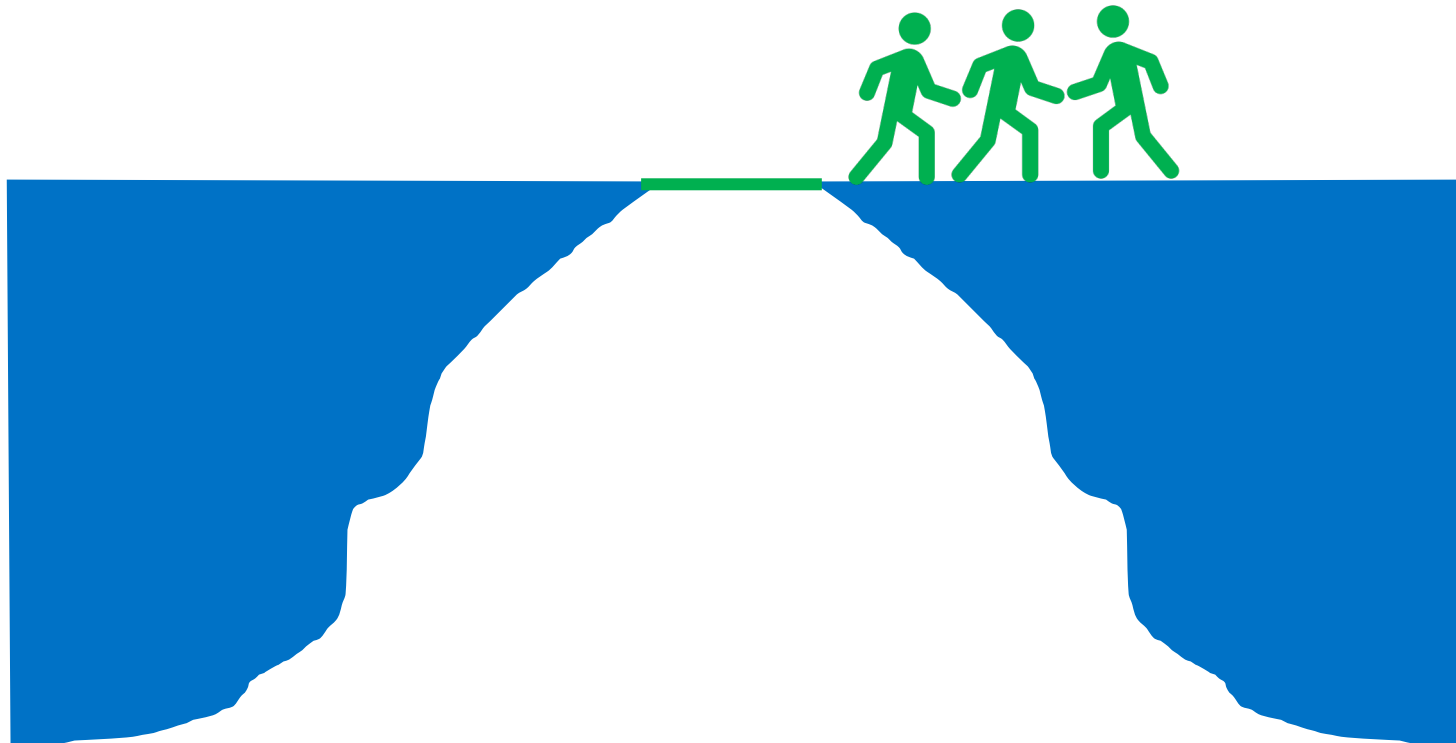
Avoid Mistakes In The Future



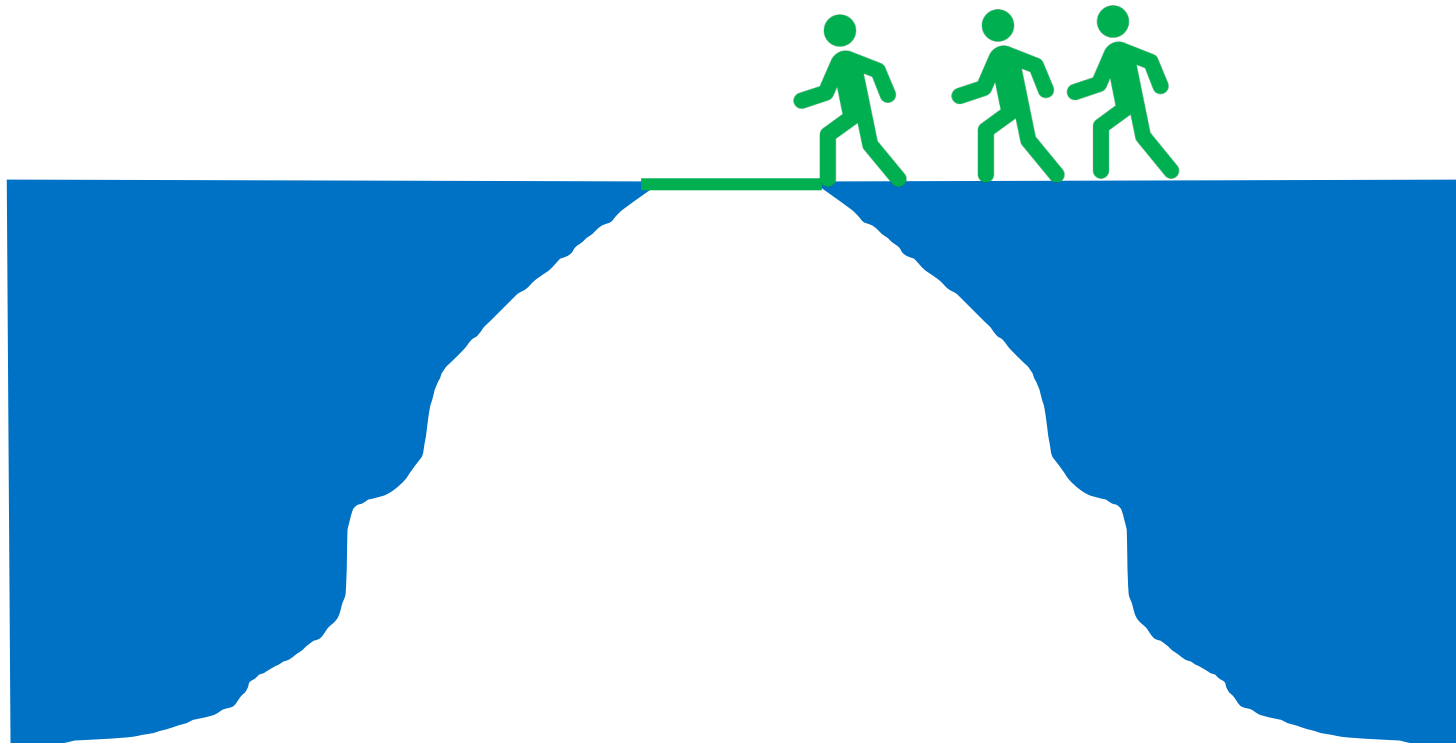
Avoid Mistakes In The Future



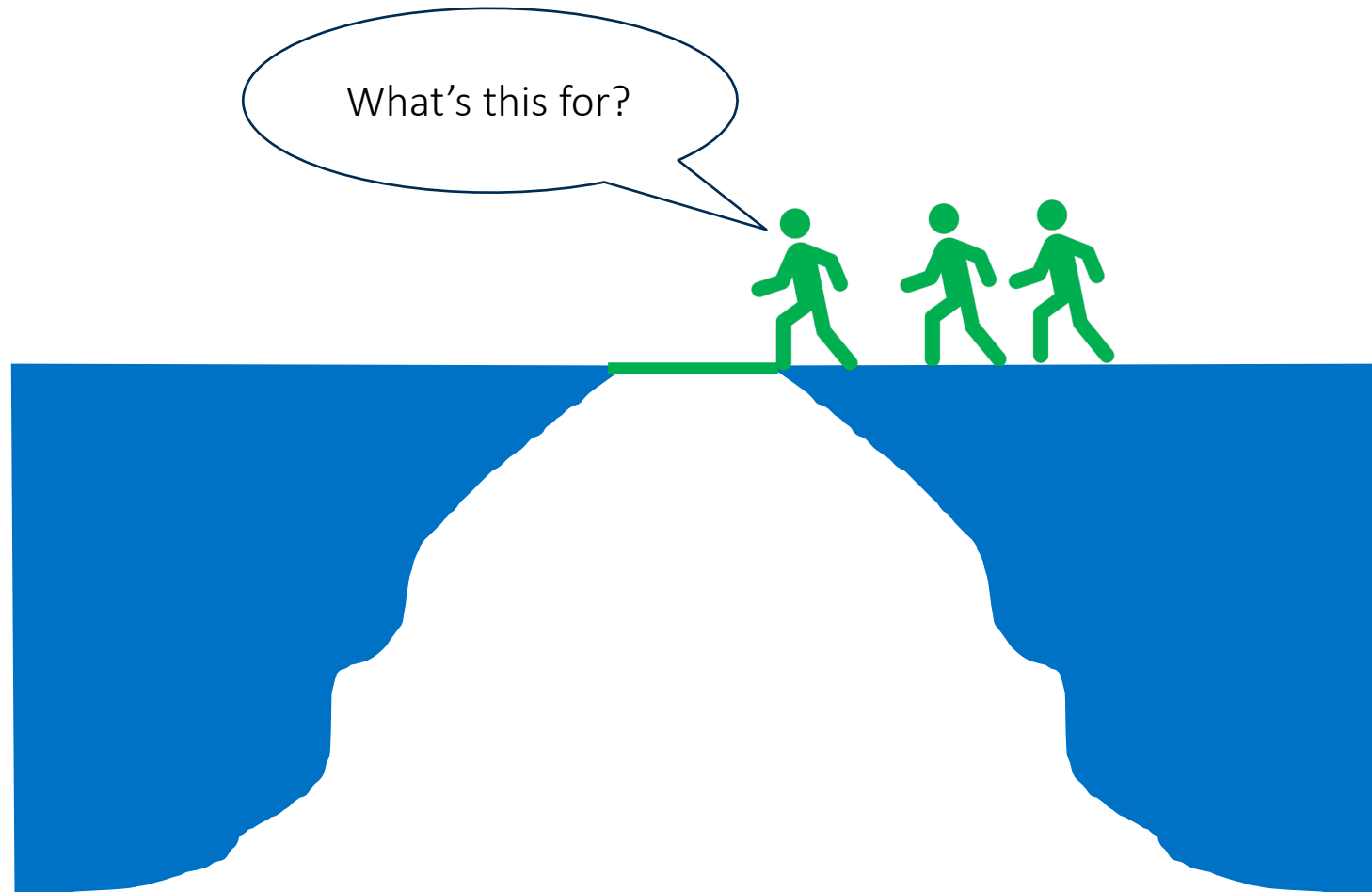
Avoid Mistakes In The Future



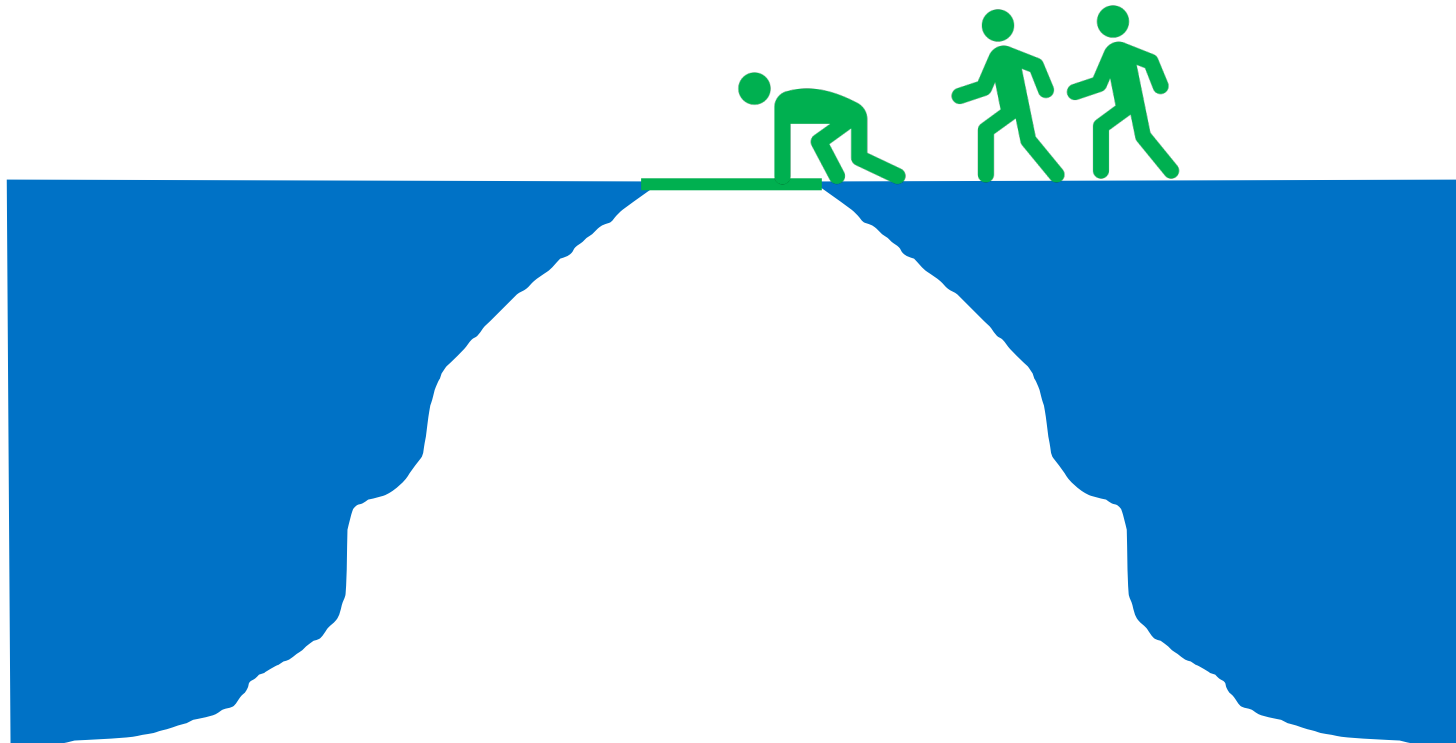
Avoid Mistakes In The Future



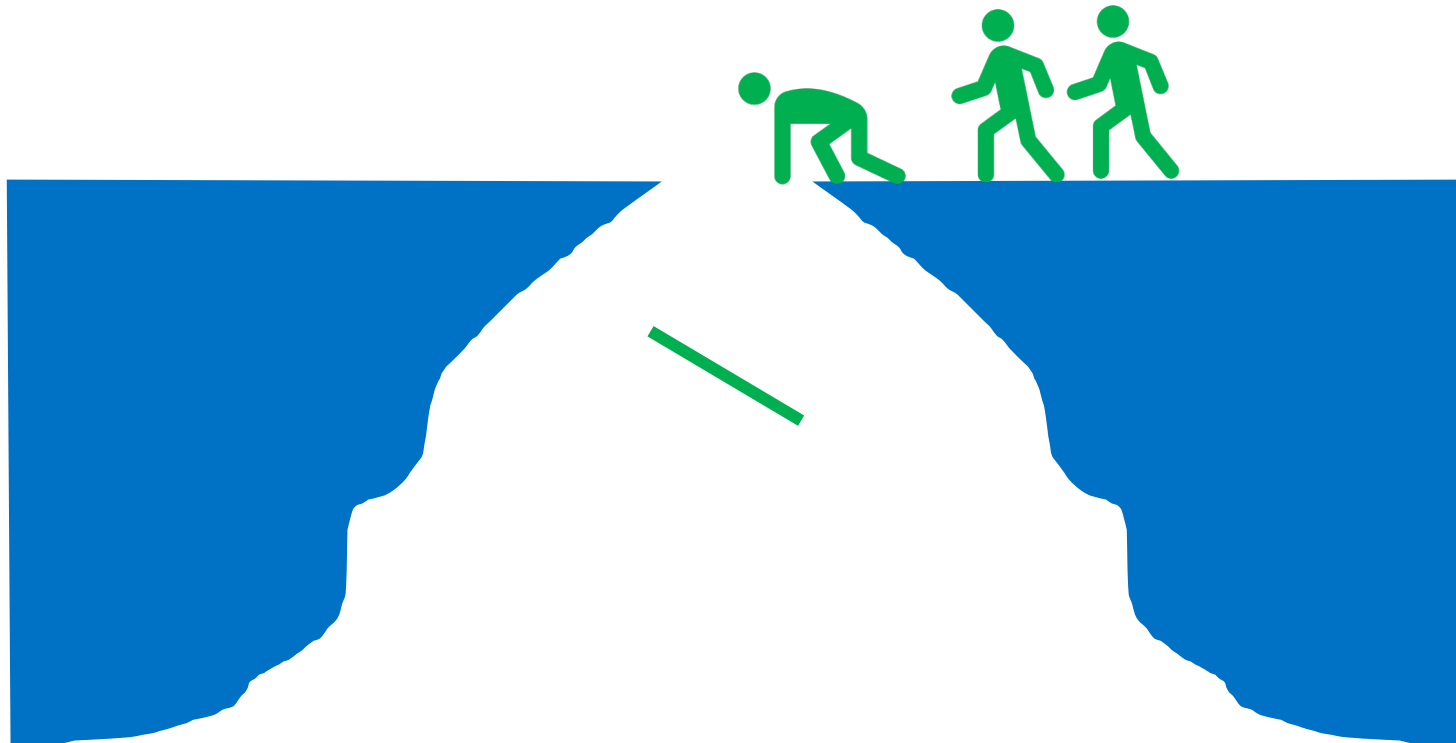
Avoid Mistakes In The Future



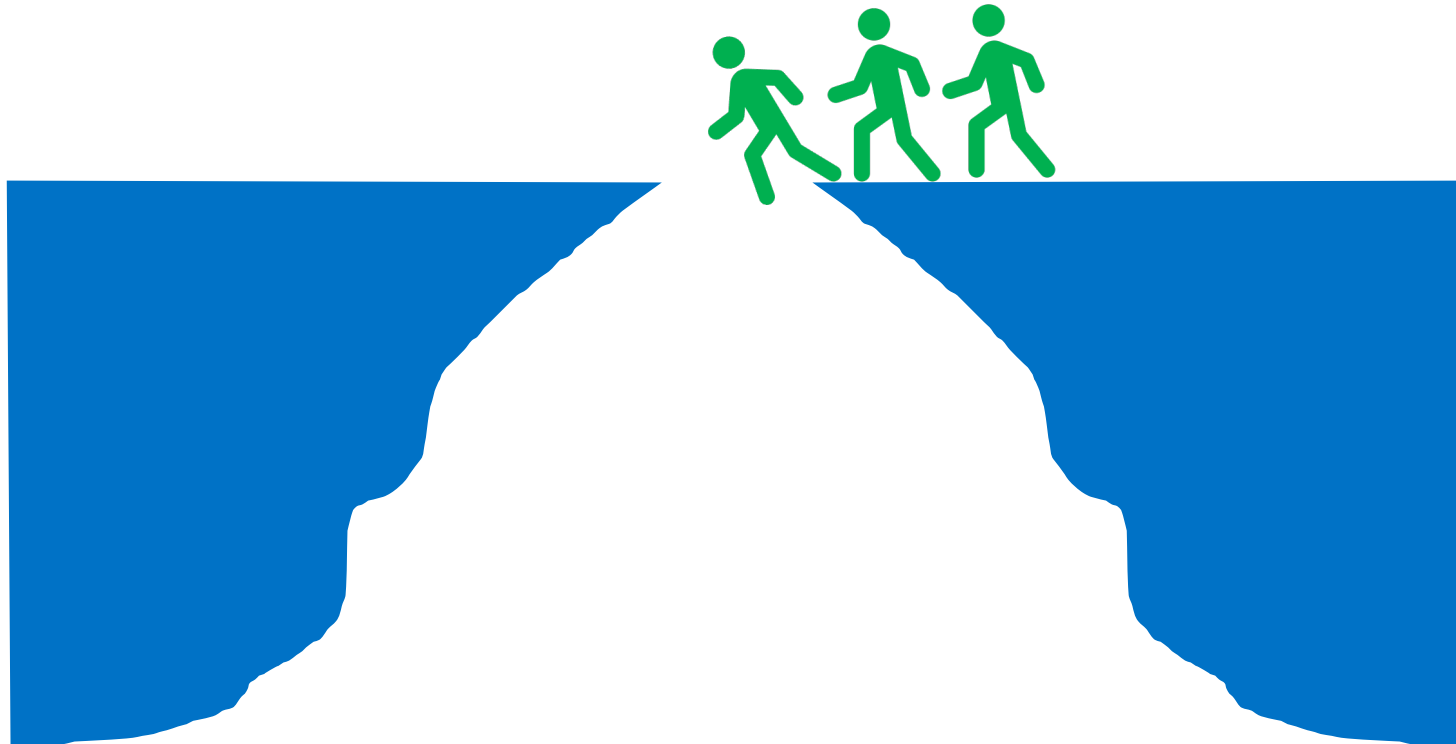
Avoid Mistakes In The Future



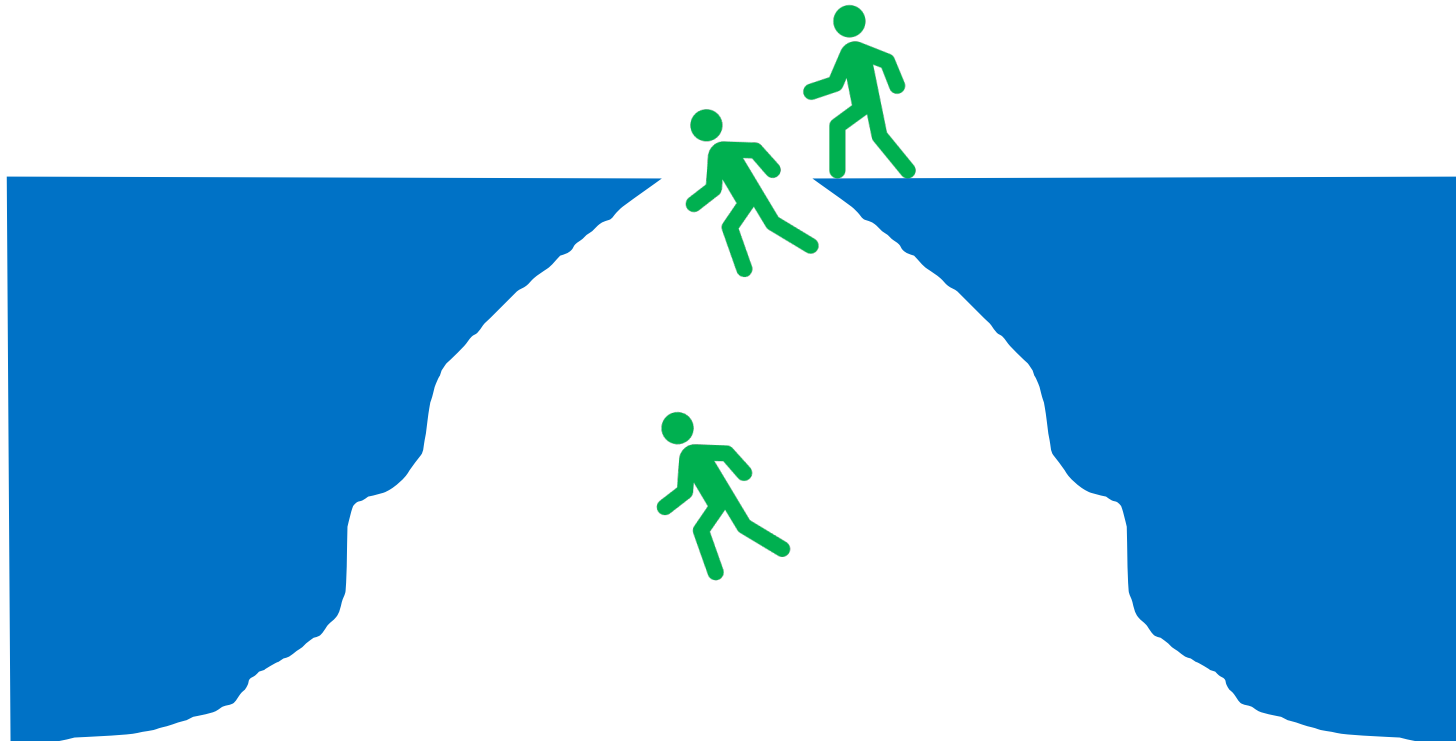
Avoid Mistakes In The Future



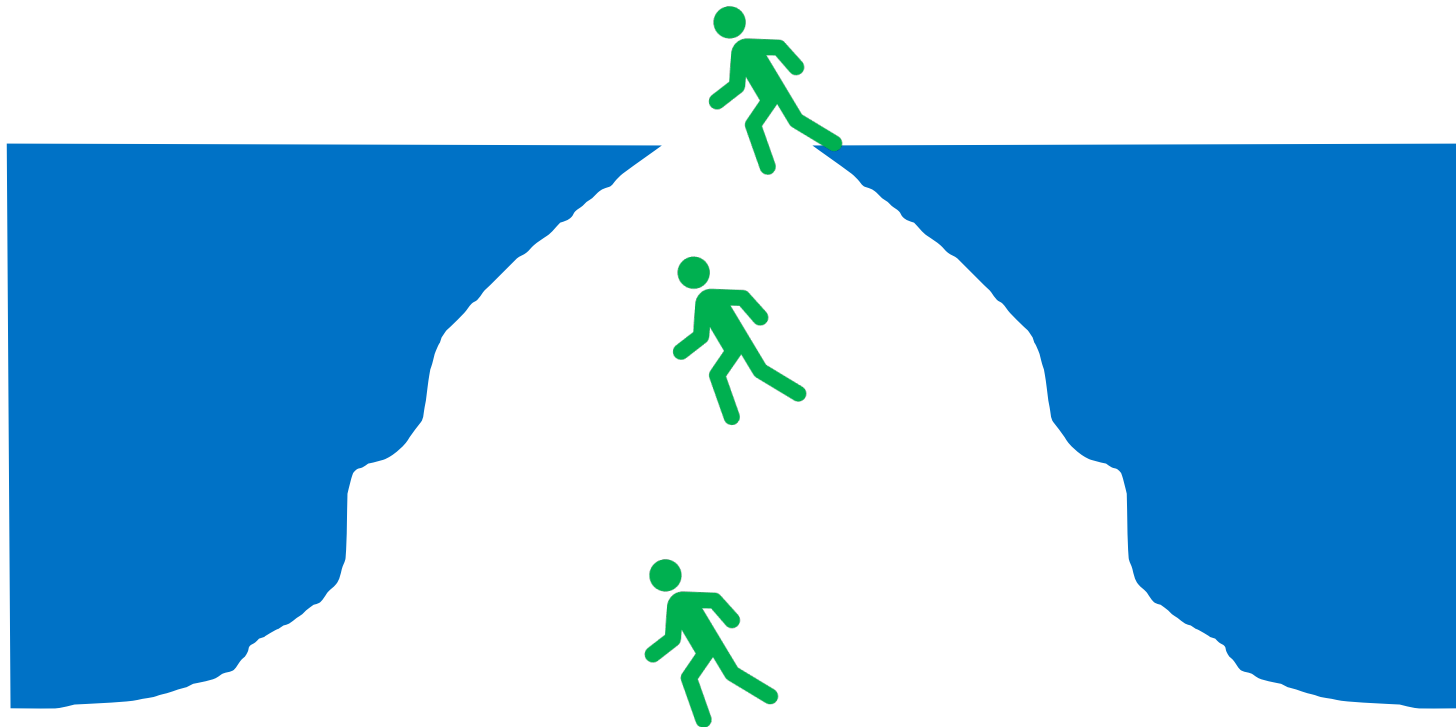
Avoid Mistakes In The Future



Avoid Mistakes In The Future



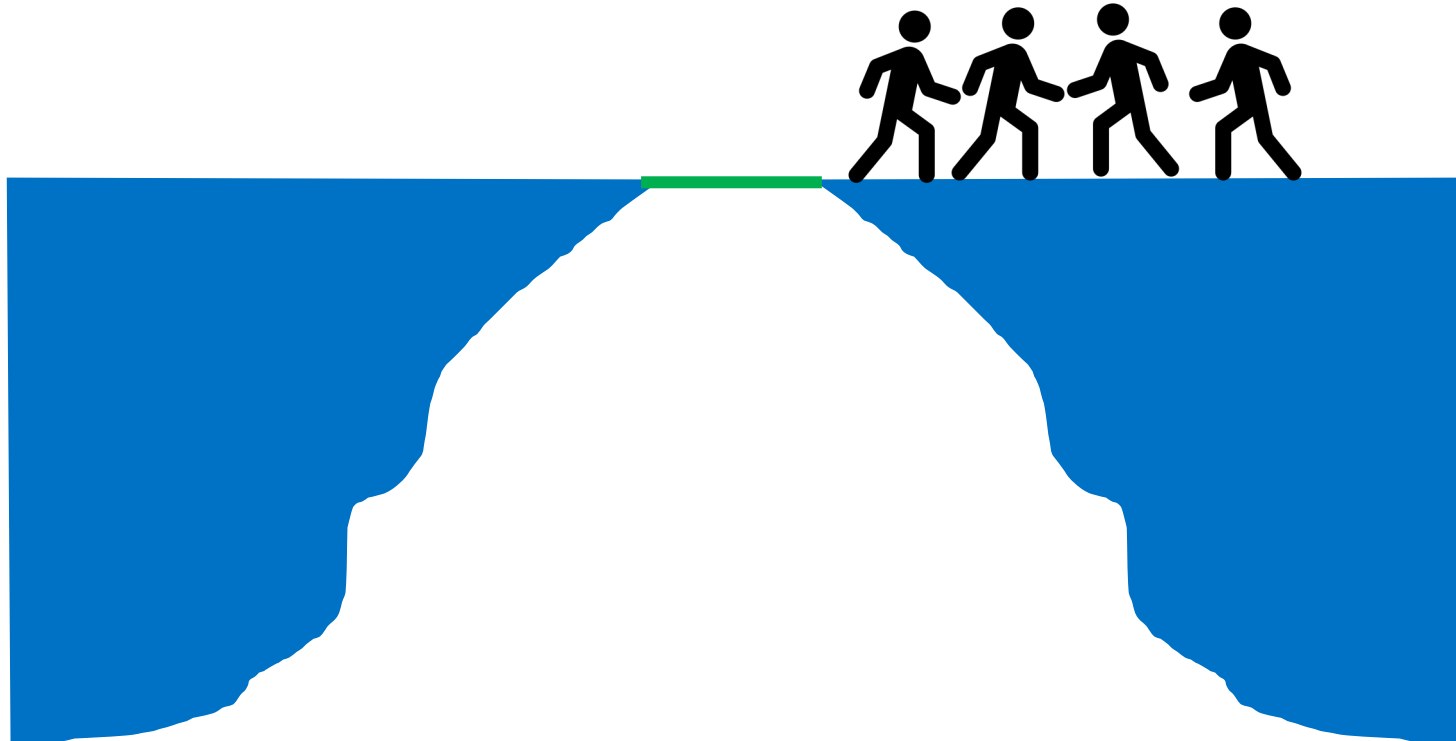
Avoid Mistakes In The Future



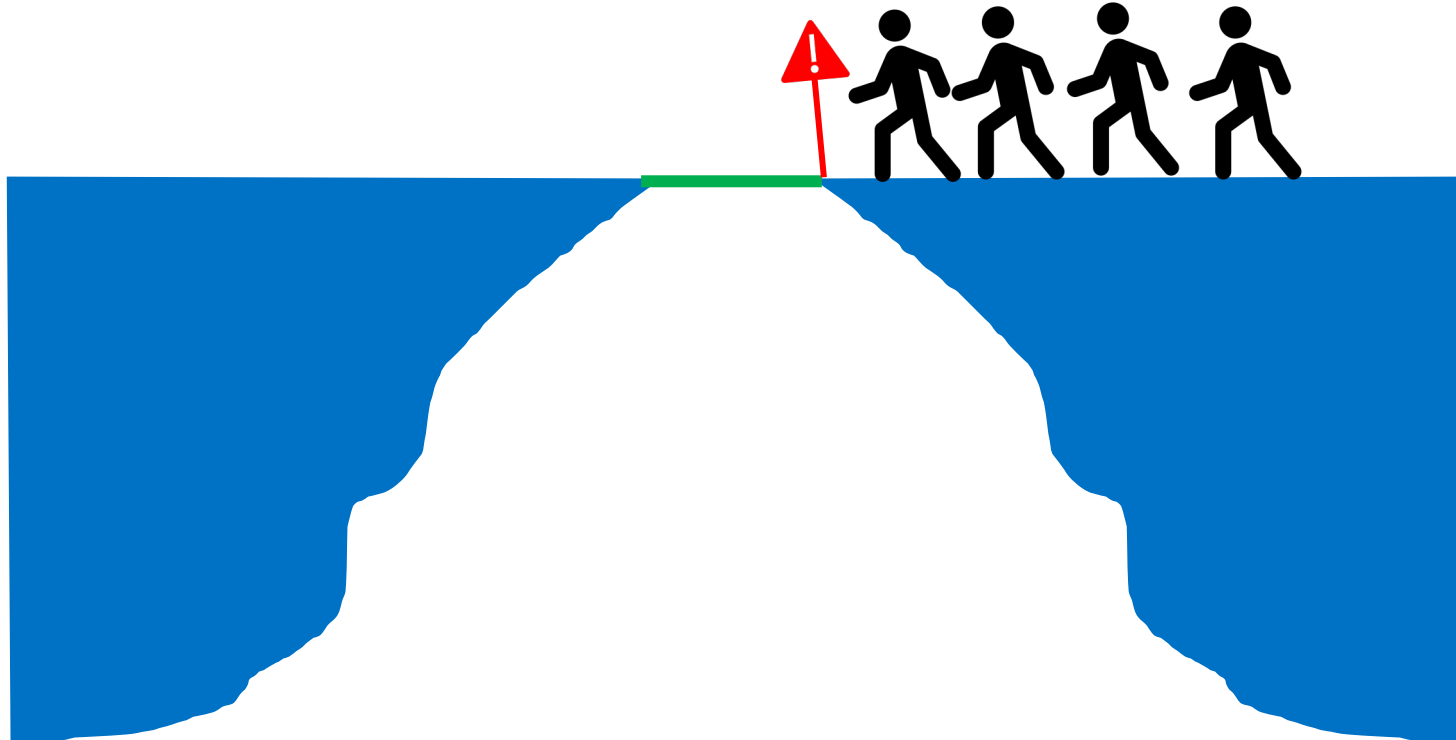
Avoid Mistakes In The Future



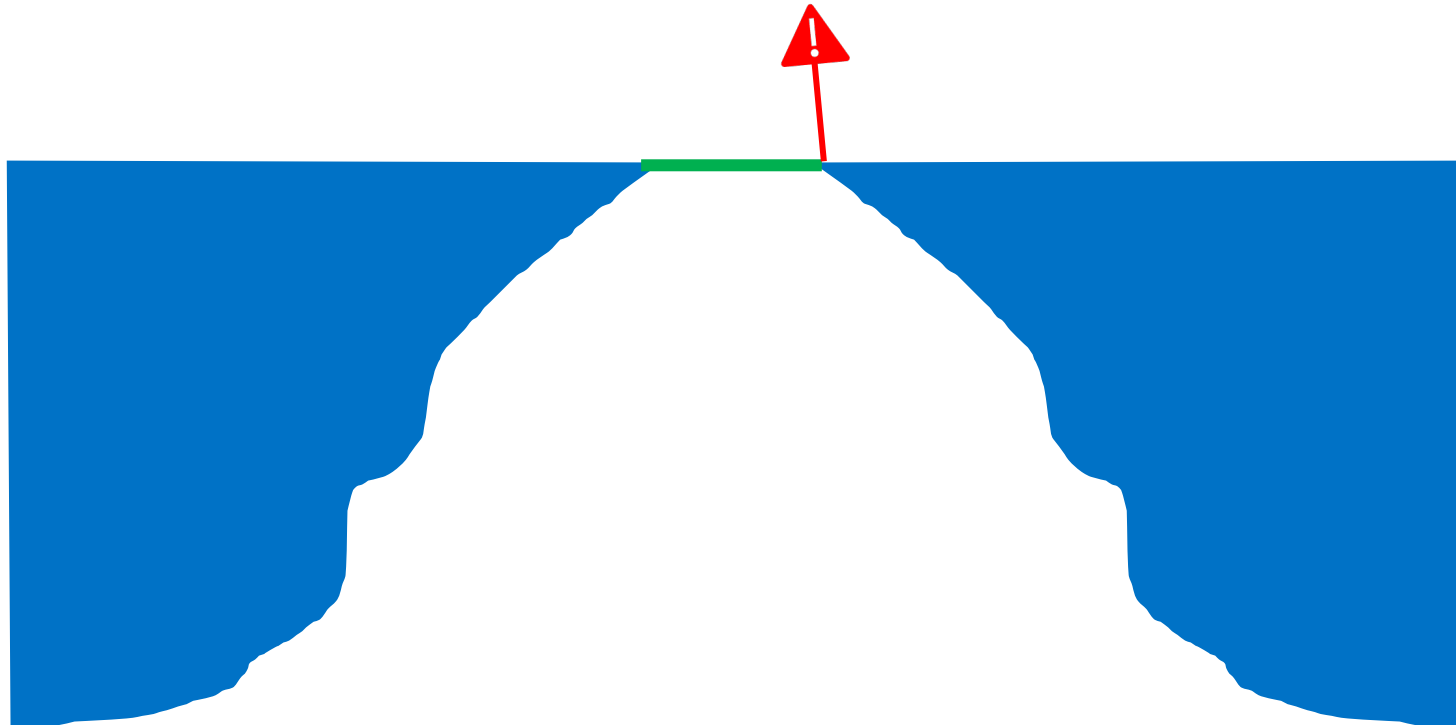
Avoid Mistakes In The Future



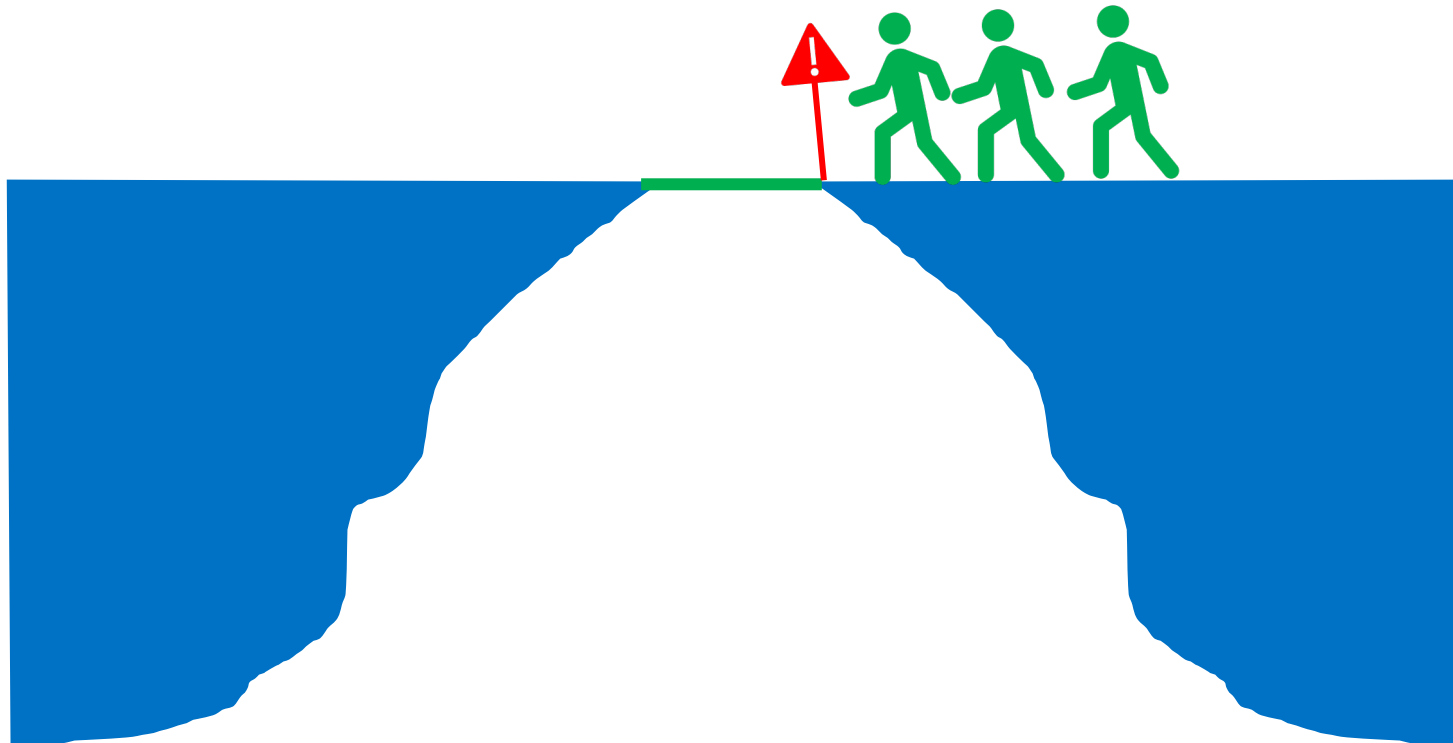
Avoid Mistakes In The Future



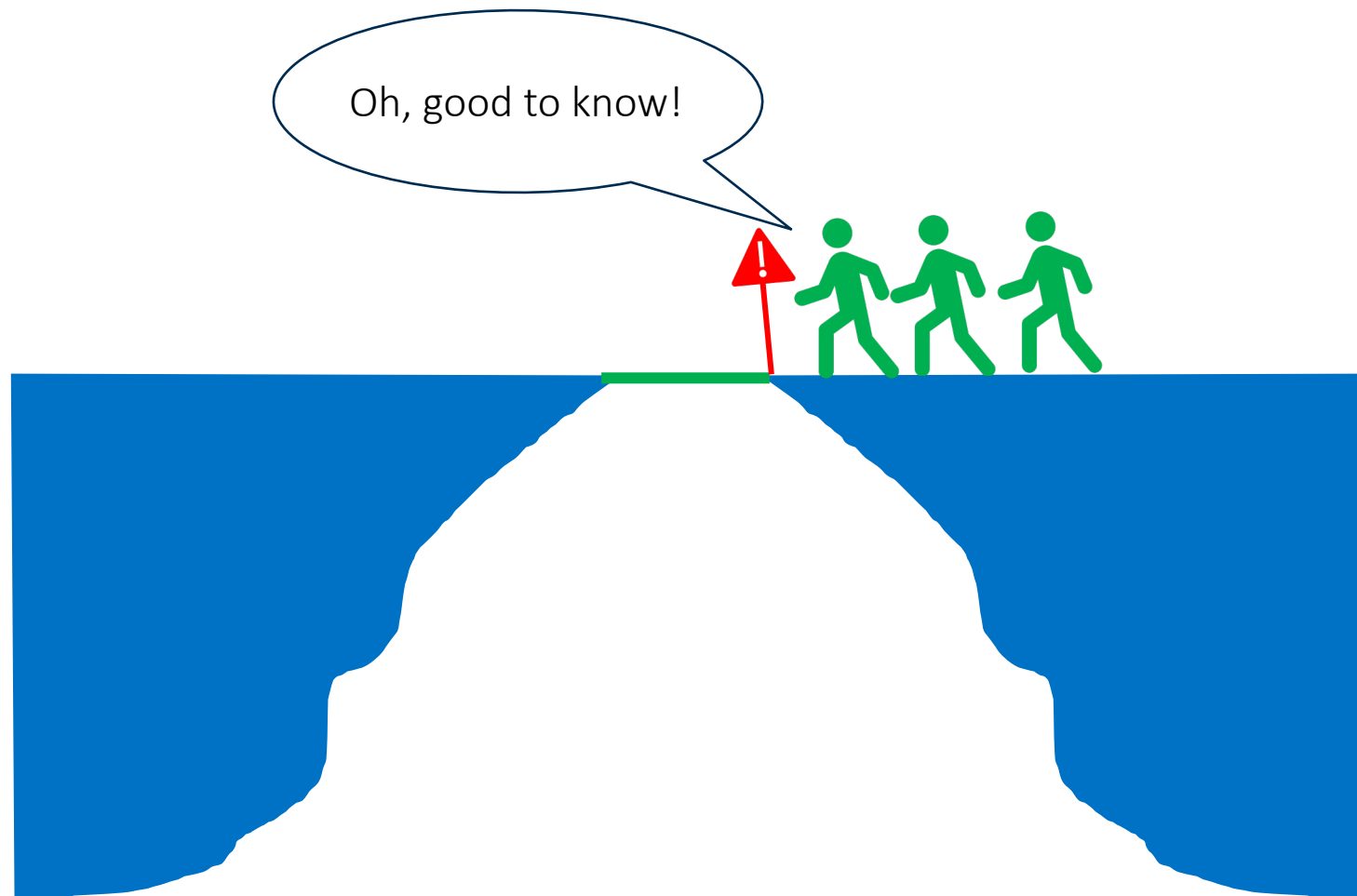
Avoid Mistakes In The Future



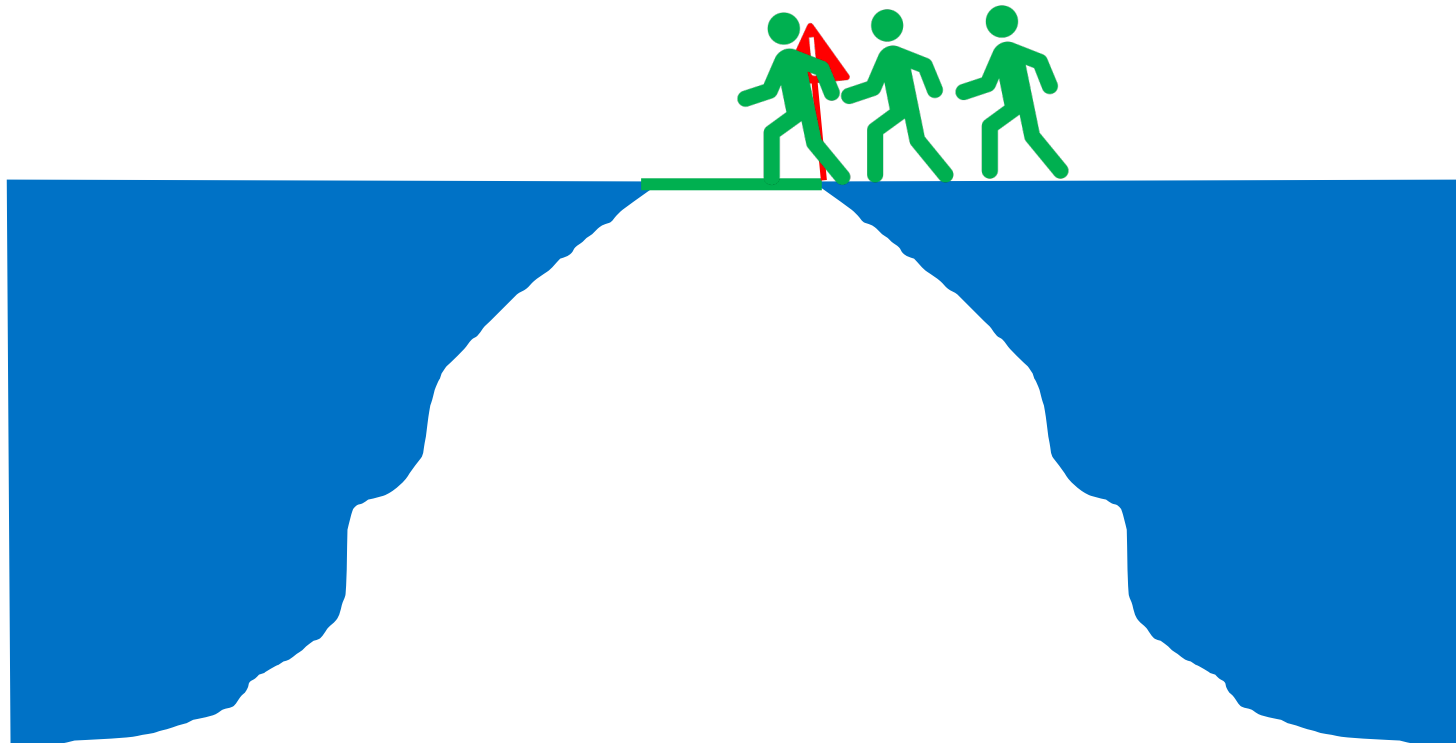
Avoid Mistakes In The Future



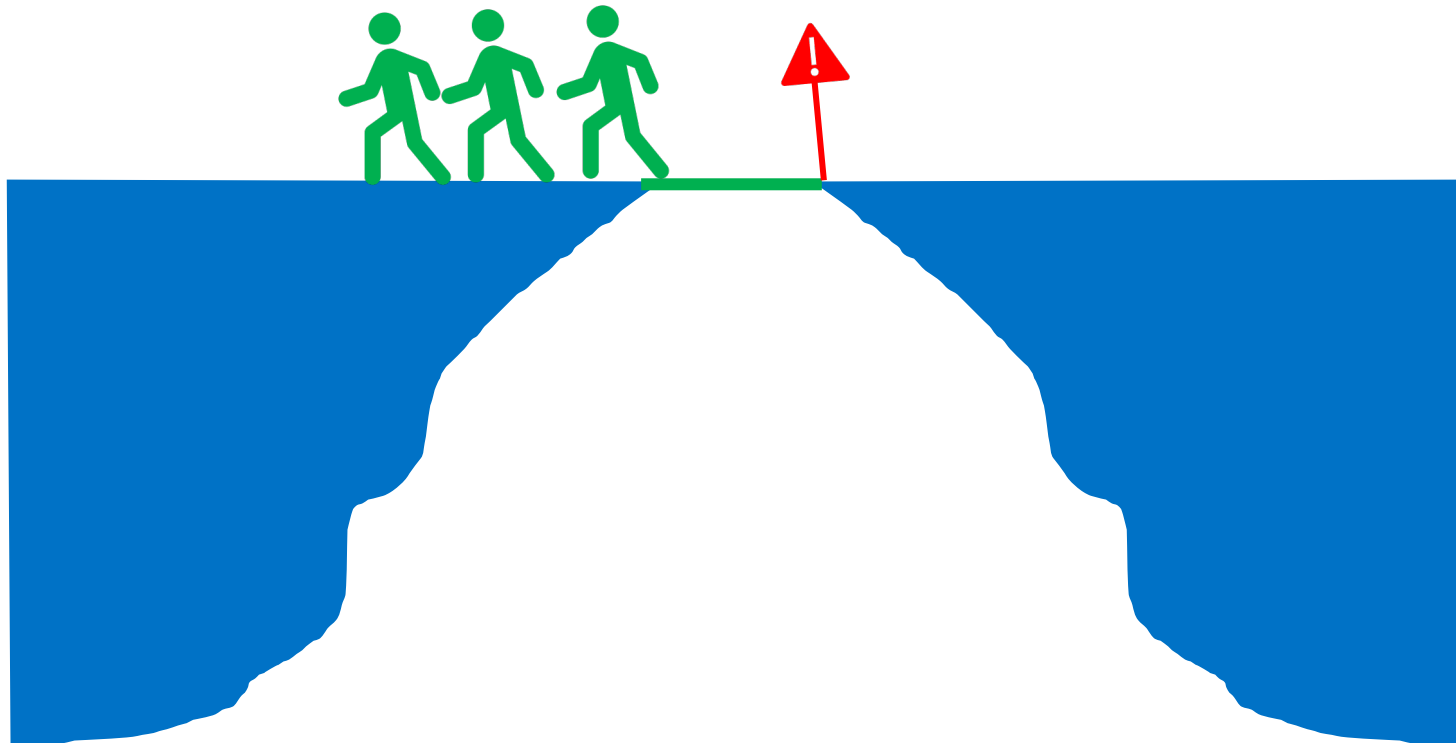
Avoid Mistakes In The Future



Avoid Mistakes In The Future



Avoid Mistakes In The Future



Architecture Decision Records (ADR)

ADR.md

Title: Avoid Implementing Feature 'Awaken Balrog'

Status: Accepted

Context:

In deciding whether to implement the Balrog Awakening feature, we draw inspiration from Gandalf's advice in Moria.

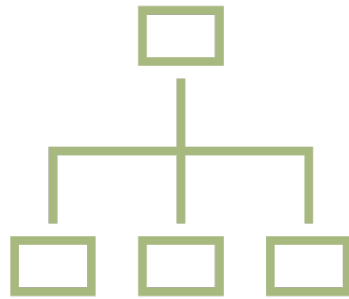
Decision:

We will not implement the Balrog Awakening feature to avoid potential catastrophic issues, such as losing our only wizard.

Consequences:

Every time we've awaken a Balrog in the past, we've lost a perfectly good wizard.

Architecture Decision Records (ADR)



Git



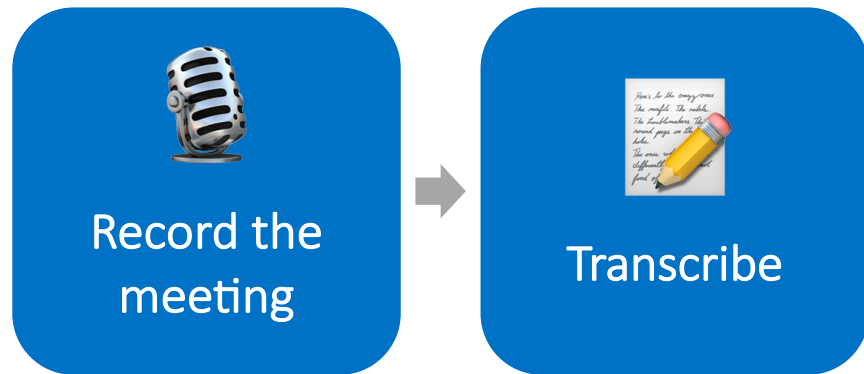
Wiki

ADRs Without the Busywork

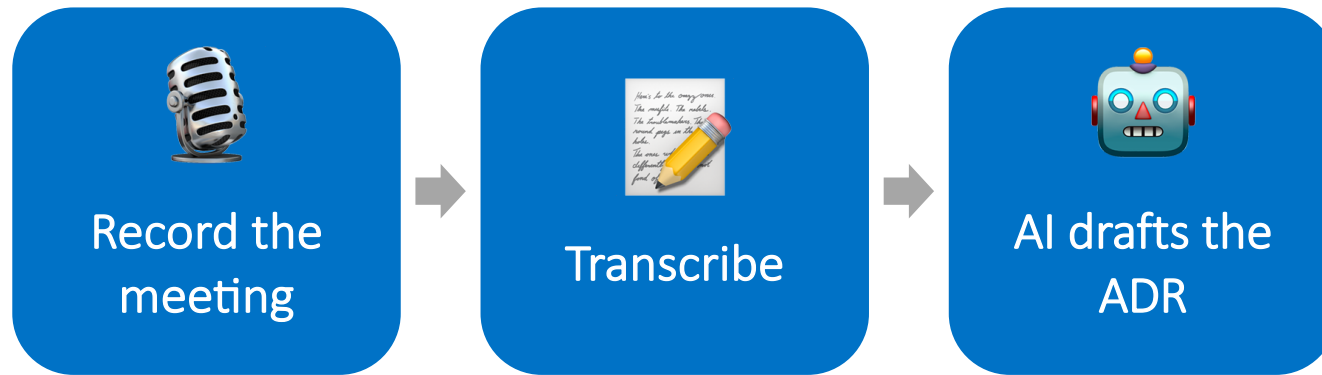


Record the
meeting

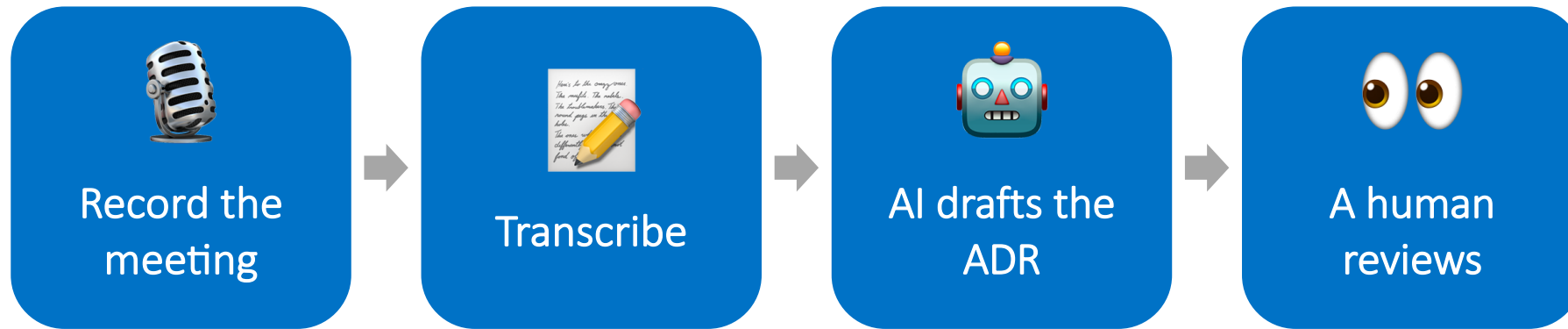
ADRs Without the Busywork



ADRs Without the Busywork



ADRs Without the Busywork



ADRs Without the Busywork



Agile / Iterative Development

Rule 2

Agile / Iterative Development

- Implement **as little as possible** to solve the problem (**MVP**)
- **Iterate** quickly with **feedback**

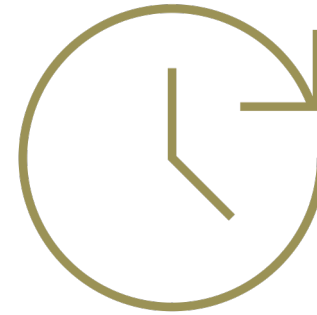
YAGNI (You Ain't Gonna Need It)

Rule 3

YAGNI



Implement Only
Current Requirements



Don't Build For
Future Needs

YAGNI in the Age of AI

- Adding later: **easier than ever**

YAGNI in the Age of AI

- Adding later: **easier than ever**
- Refactoring: **easier than ever**

YAGNI in the Age of AI

- Adding later: **easier than ever**
- Refactoring: **easier than ever**
- Build it **when you need it**

KISS (Keep It Simple, Software-developer)

Rule 4

KISS

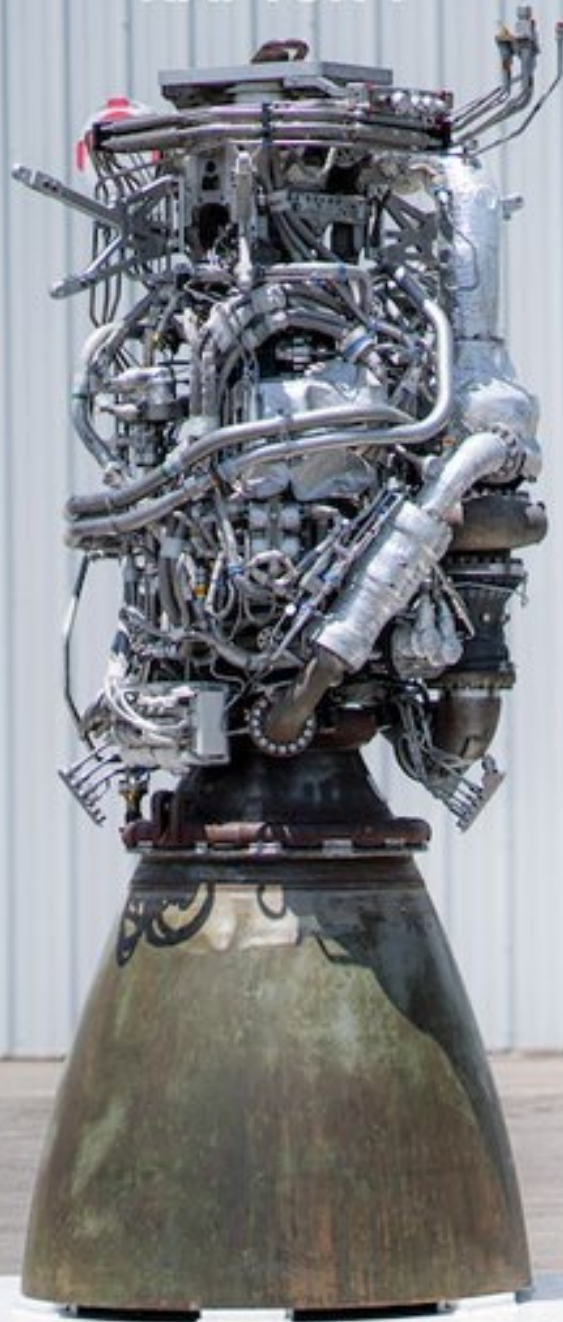


Less Surface Area
For Failures



Easier to
Maintain

RAPTOR 1



RAPTOR 2

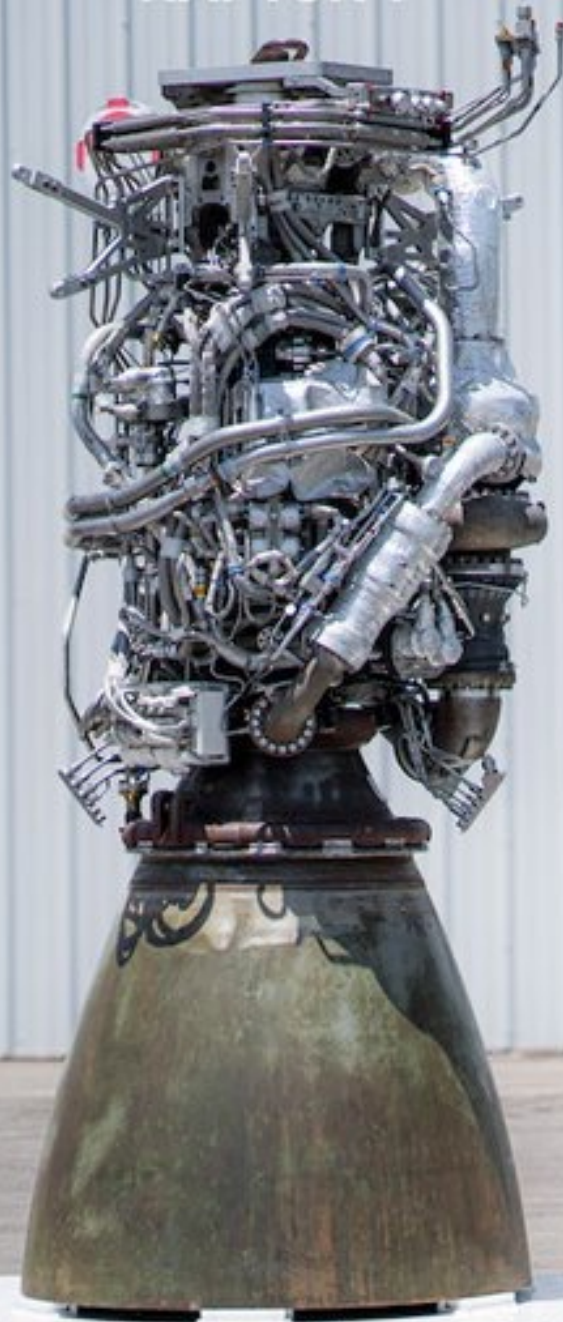


RAPTOR 3



TRAILHEAD
TECHNOLOGY PARTNERS

RAPTOR 1



RAPTOR 2



RAPTOR 3

Most reliable
Most thrust
Least weight
Fewest parts
Cheapest



TRAILHEAD
TECHNOLOGY PARTNERS

KISS

"I would have written a shorter letter,
but did not have the time."

- Blaise Pascal

5 DRY but in a WET Way

Rule 5

DRY but in a WET Way

DRY - Don't Repeat Yourself

WET - Write Everything Twice

Principle of Least Astonishment (POLA)

Rule 6

Principle of Least Astonishment

"A component of a system should behave in a way that most users will expect it to behave, and therefore not astonish or surprise users."

Principle of Least Astonishment (POLA)



Reduces
Cognitive Load



Reduces
Mistakes



Easier to
Learn

Pull Reqs / Code Reviews

Rule 7

Pull Reqs / Code Reviews



Detecting Patterns
of Over-
Engineering



Catching
Unnecessary
Features Early



Ensuring
Adherence to Best
Practices



Encouraging
Incremental
Development

“Shift Left”

The **earlier** in the development lifecycle you **catch a defect**, the **less expensive** it is to fix.

Typical Code Review Checklist

- Bug Check
- Acceptance Criteria Check
- Code Standards Check
- Clarity Check
- Performance Check
- Documentation Check

Typical Code Review Checklist

- Bug Check
- Acceptance Criteria Check
- Code Standards Check
- Clarity Check
- Performance Check
- Documentation Check
- ✓ Simplicity Check
- ✓ YAGNI Check
- ✓ KISS Check
- ✓ Premature Optimization Check

DEMO

Catching Over-Engineering

Feedback Loops

Rule 8

Feedback Loops



Aligning Development
with User Needs

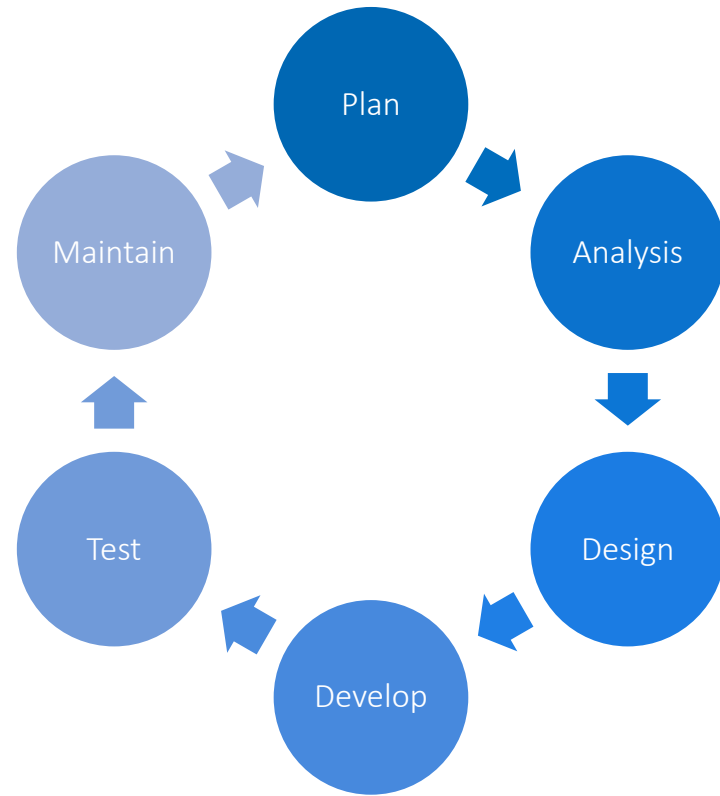


Encouraging Iterative
Development

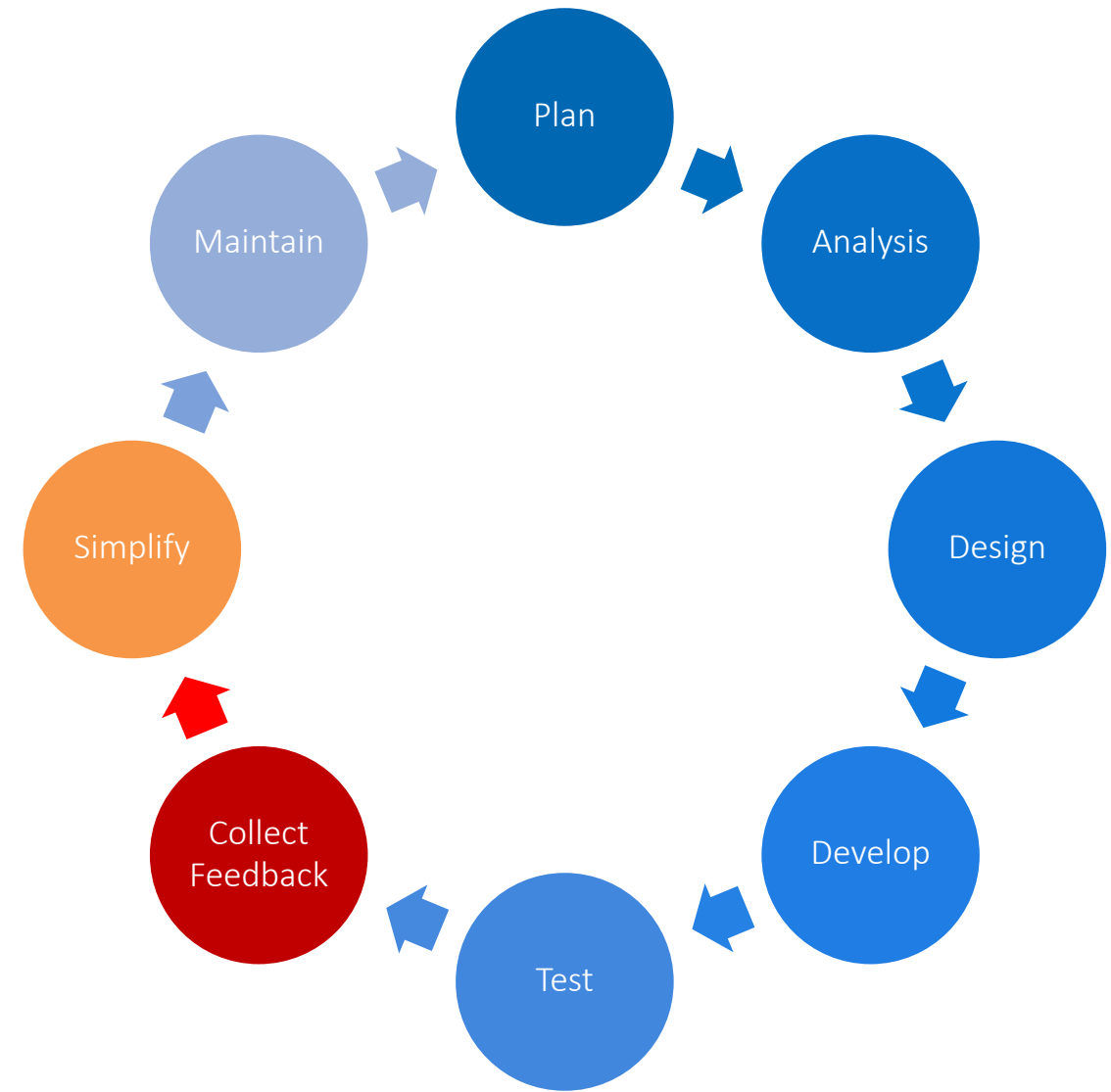
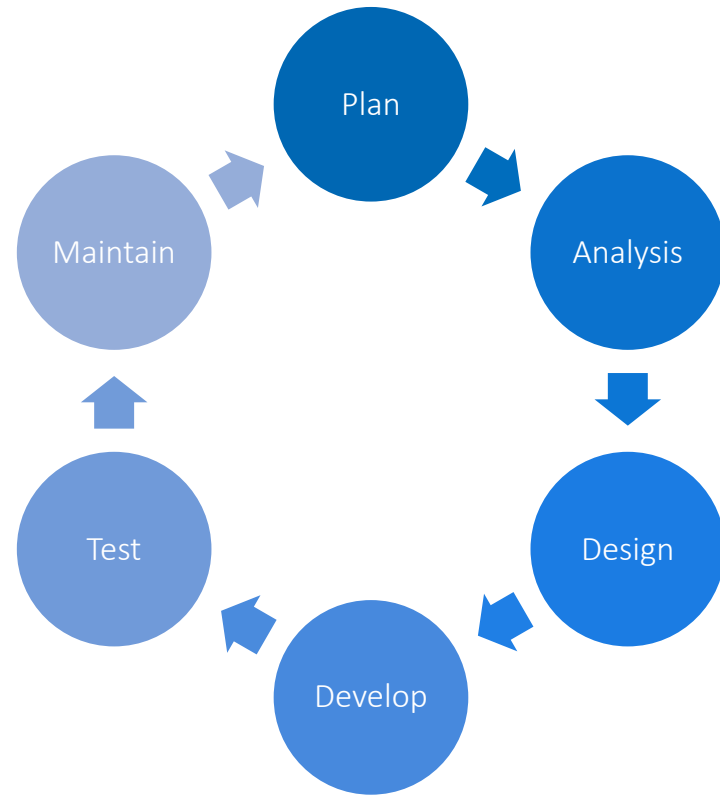


Detecting and correcting
over-engineering early

Feedback Loops



Feedback Loops



Adopt Proven Solutions

Rule 9

Adopt Proven Solutions



Adapting Proven Solutions



More predictable
outcomes



Reduces unknowns



Manages risk

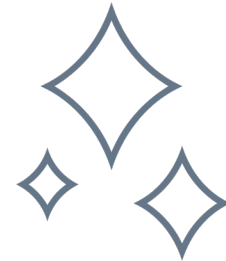
Adapting Proven Solutions



Use Frameworks
You Know



New Architectures
Only as POC



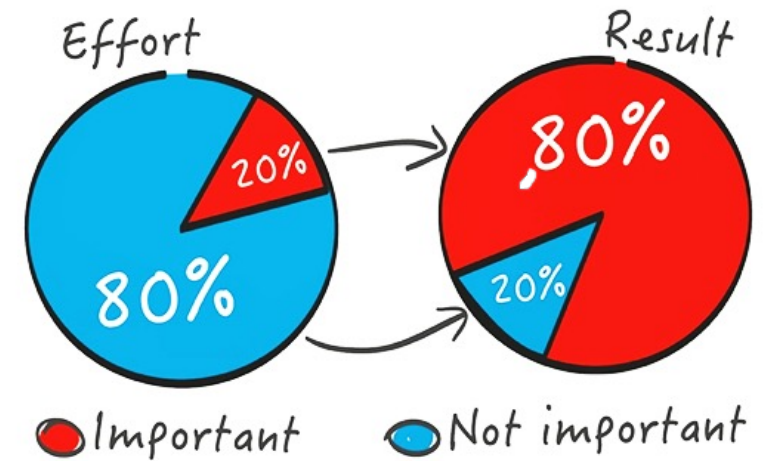
Adopt New Tech Behind
Buzzword Curve

80/20 Rule (Pareto Principle)

Rule 10

80/20 Rule

"Roughly 80% of consequences come from 20% of causes."

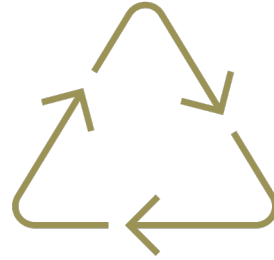


SOURCE <https://www.sreedeeep.com/the-pareto-principle-explained/>

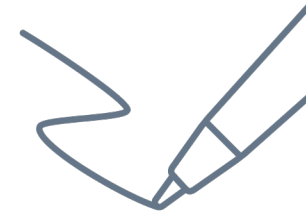
80/20 Rule Allows



Focusing on
high-impact features



Efficient
use of resources



Avoid
perfectionism

Summing Up

1. Over-engineering is **common** but **costly**
2. **Know the signs** of over-engineering
3. Implement **rules to avoid** over-engineering



Thanks! Questions?

Jonathan "J." Tower

Founding Partner

🏆 13x Microsoft MVP in .NET & AI

⚙️ VP of .NET Foundation

🎙️ Blue Blazes podcast host

